

Kubernetes (K8s) Basics

What you will build

In this workshop, you will start by working with Docker Compose to manage multi-container setups, then deploy and explore the Pedelec server-side system. You'll learn to write Kubernetes manifests, interact with clusters using Rancher and kubectl, and create Deployments, Services, and Ingresses. By the end, you will have deployed a microservice architecture to the cloud and gained hands-on experience with scalable, containerized application management.

Prerequisites

- Containers
- Images
- Docker
- UML
- CLI

Key Vocabulary

- Compose File
- Kubernetes
- kubectl
- Manifest
- Deployment
- Service
- Ingress
- Helm

Who are we?



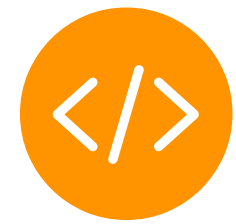
Matthias Linhuber



Doctoral Student at TUM



Educator at heart



Software and Infrastructure Architect



Research Areas

- Container-based Software Engineering
- Infrastructure Orchestration
- Scaling Education Technology



Pre Survey

I need your help!



<https://go.tum.de/910287>

Learning Goals

- **Context and assumptions**
 - You understand the basics of Docker
 - You understand the ISO/OSI Model
 - You know how to use the CLI
- **At the end of this session you are able to**
 - Understand the key features of docker compose
 - Create and use custom docker compose setups
 - Understand the basic concepts of Kubernetes
 - Use kubectl and helm to interact with Kubernetes

Docker Compose

What is Docker Compose?

- Compose is a layer “on top” of Docker
- Docker compose allows to
 - **Combine** multiple containers that interact with each other
 - Start / Stop multiple containers easily
 - **Persist** data in volumes
 - Configure multiple **isolated environments** (e.g. development and testing)
 - **Deploy** software to a Server / VM

The Docker Compose File

- Docker compose files are written in **yaml**: `compose.yaml`
- Defining:
 - **services**
 - Abstraction of a computing resource
 - A valid `compose.yaml` has to specify at least one service
 - **networks**
 - Abstraction of a communication channel
 - Handle service discovery
 - `volumes / configs / secrets`

Example: services

- Create a `compose.yml` file:

```
services:  
  app:  
    image: "hello-world"
```

We create a service called "app"

From the image "hello-world"

- Start the compose with

```
~/workshop> docker compose up
```




Example: Multiple services

- Update the `compose.yml` file:

```
services:  
  app1:  
    image: "nginxdemos/hello"  
  app2:  
    image: "nginxdemos/hello"
```

- Start the compose with

```
~/workshop> docker compose up -d
```

The containers now run in the background ("detached")

```
~/workshop> docker compose ps
```

Shows the running containers



docker compose ps

- Initial version of our compose.yaml

```
~/workshop> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
workshop-serverside1-1	nginxdemos/hello	"/docker-entrypoint..."	serverside1	3 minutes ago	Up 3 minutes	80/tcp
workshop-serverside2-1	nginxdemos/hello	"/docker-entrypoint..."	serverside2	3 minutes ago	Up 3 minutes	80/tcp

The containers offer port 80 - but we can't access it yet... 😞

```
~/workshop> docker compose down
```



Example: Make **services** accessible

- Update the `compose.yml` file:

```
services:
  app1:
    image: "nginxdemos/hello"
    ports:
      - "8081:80"
  app2:
    image: "nginxdemos/hello"
    ports:
      - "8082:80"
```

We map the host port 8081 to the container port 80

- Start the compose



docker compose ps

- Initial version of our compose.yaml

```
~/workshop> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
workshop-serverside1-1	nginxdemos/hello	"/docker-entrypoint...."	serverside1	3 minutes ago	Up 3 minutes	80/tcp
workshop-serverside2-1	nginxdemos/hello	"/docker-entrypoint...."	serverside2	3 minutes ago	Up 3 minutes	80/tcp

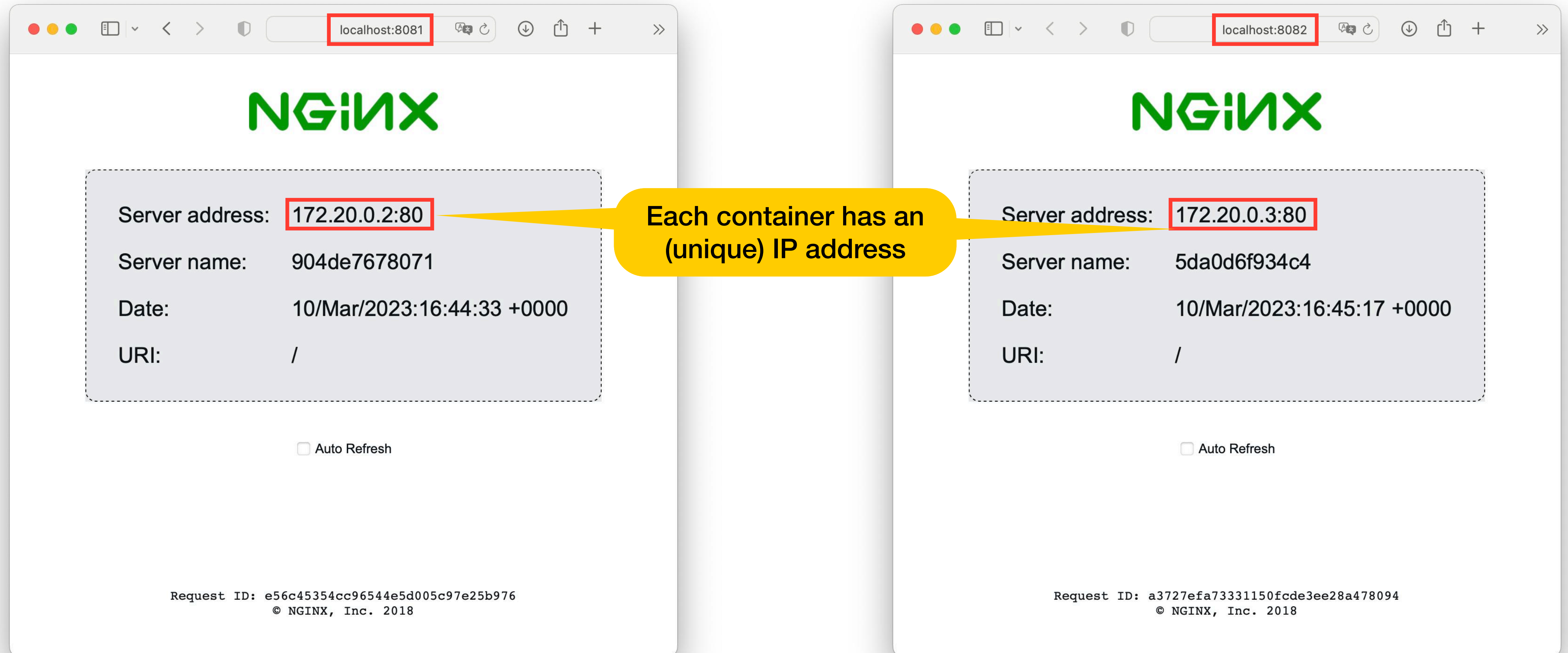
- Updated compose.yaml with port mappings

```
~/workshop> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
workshop-serverside1-1	nginxdemos/hello	"/docker-entrypoint...."	serverside1	2 minutes ago	Up 2 minutes	0.0.0.0:8081->80/tcp
workshop-serverside2-1	nginxdemos/hello	"/docker-entrypoint...."	serverside2	2 minutes ago	Up 2 minutes	0.0.0.0:8082->80/tcp

Now the container port is forwarded
(accessible) to the host 😊
See <http://localhost:8082>

Example: Inspect our services



The image displays two browser windows side-by-side, each showing the NGINX status page. The left window's address bar is `localhost:8081` and the right window's is `localhost:8082`. Both windows show a status box with the following information:

Field	Left Window (localhost:8081)	Right Window (localhost:8082)
Server address:	172.20.0.2:80	172.20.0.3:80
Server name:	904de7678071	5da0d6f934c4
Date:	10/Mar/2023:16:44:33 +0000	10/Mar/2023:16:45:17 +0000
URI:	/	/

A yellow callout bubble points to the IP addresses in both windows, containing the text: "Each container has an (unique) IP address".

At the bottom of each window, there is an "Auto Refresh" checkbox and a "Request ID" (e56c45354cc96544e5d005c97e25b976 for the left, a3727efa73331150fcde3ee28a478094 for the right), along with the copyright notice "© NGINX, Inc. 2018".

Configure Containers: environment

- Inject variables into containers

```
services:  
  app1:  
    image: "nginxdemos/hello"  
    ports:  
      - "8080:80"  
    environment:  
      - "SOME_KEY=SOME_VALUE"
```

Never put secrets directly into the compose file - Use .env files instead

- Applications need to handle environment variables!

Our Running Example: Pedelec App

Our office is located in close proximity to our customers. The access to public transportation is poor and the number of available parking spots at the customer's site is low.

With a bike, the customer site is easy to reach and only minimal parking space is required. To reduce the commuting time to a minimum we plan to operate a pool of Pedelecs (electricity powered bicycles) for our employees. Pedelecs can be reserved with a new mobile app.



Our Running Example: Pedelec App

Marie, an Accenture employee in Munich, has an appointment at BMW's "Forschungs- und Innovationszentrum". BMW does not have enough public parking spots so she cannot use her car. If she uses a Pedelec the trip can be done in 10 minutes.

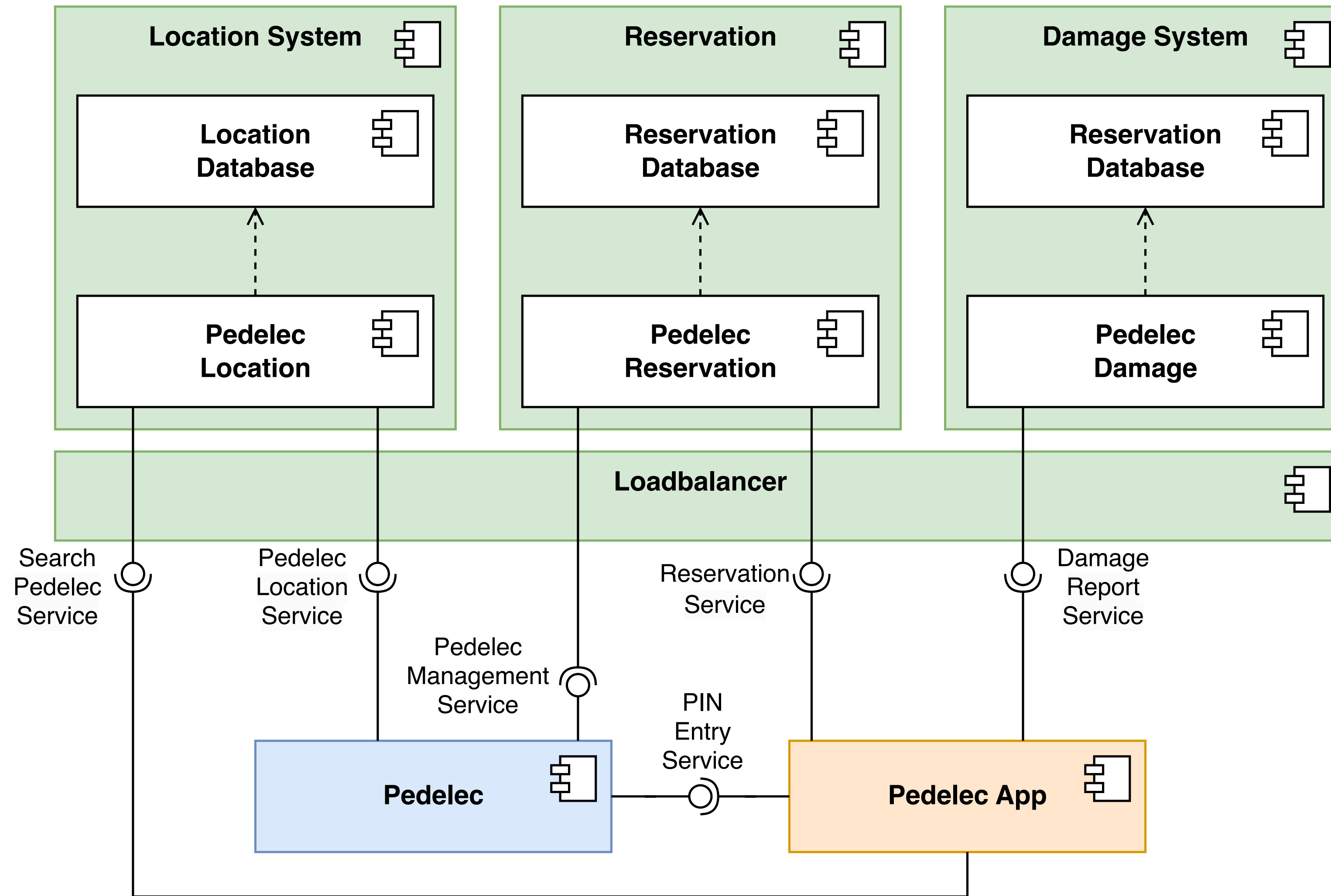
She searches for a Pedelec in her app and sees that 3 of them are available and have enough battery power to reach BMW. Marie reserves this Pedelec.

When Marie arrives at the bike lot, the app automatically displays the PIN to unlock the Pedelec. She unlocks it and drives to BMW. There she notices that the bell of the bike is damaged. She reports this damage with the app. After she is back at Accenture, she returns the bike and releases her reservation.

Software Engineering Happening



Pedelec Server Side



Exercise: Pedelec App Overview



<https://github.com/Mtze/kubernetes-workshop>

Task 1: Fork and Clone the Pedelec Repo

Task 2: Inspect the Server Side (`./exercises/01-microservices`)

Task 3: Start the Server Side with docker compose

Tasks:

```
~> git clone git@github.com:Mtze/kubernetes-workshop.git  
~> cd kubernetes-workshop  
~/pedelec-app> code .  
~/pedelec-app> cd exercises/01-microservices  
~/.../Pedelec Server> docker compose up
```

Exercise: Pedelec App API



Task 1: Open Bruno

Task 2: Open Bruno config

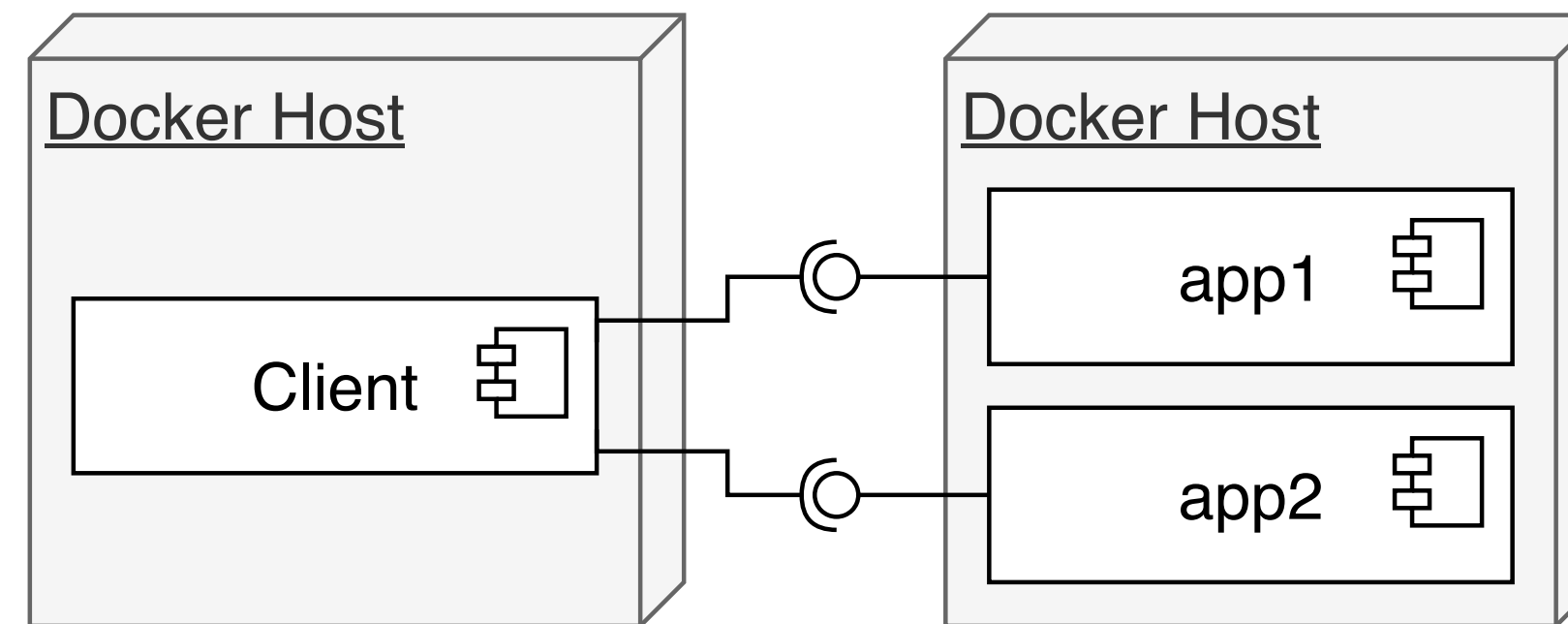
Task 3: Check if API is working

The Docker Compose File

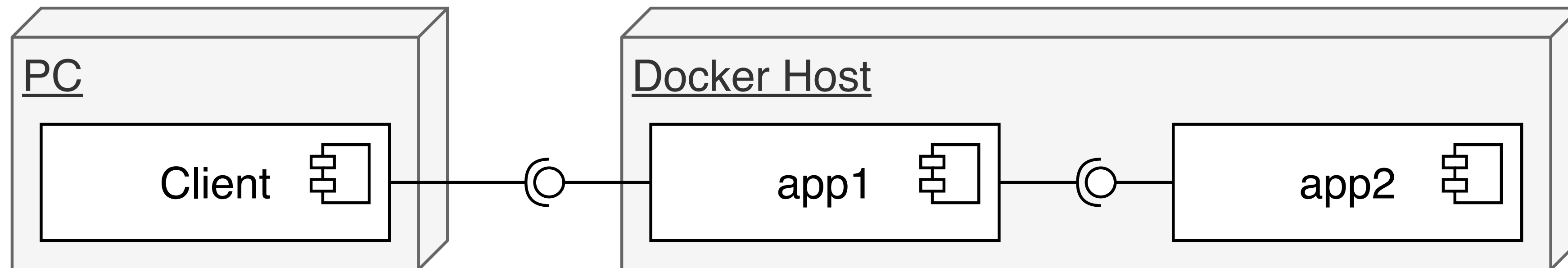
- Docker compose files are written in **yaml**: `compose.yaml`
- Defining:
 - ✓ **services**
 - Abstraction of a computing resource
 - A valid `compose.yaml` has to specify at least one service
 - **networks**
 - Abstraction of a communication channel
 - Handle service discovery
 - `volumes / configs / secrets`

Example: networks

- Architecture we created with ports



- Architectures we can create with networks





Example: Create networks

- Update the `compose.yml` file:

```
services:  
  app1:  
    image: "nginxdemos/hello"  
    ports:  
      - "8080:80"
```

We're still using ports to map the service to the host

```
networks:  
  - mynet
```

Both containers connect to the same network

```
app2:  
  image: "nginxdemos/hello"  
  networks:  
    - mynet  
  expose:  
    - "80"
```

We replaced the port mapping with an expose to limit access to within the "mynet" network

```
networks:  
  mynet:
```

We explicitly define a network



docker compose ps

- Updated compose.yaml with port mappings

app1 is reachable from outside the network: **port**

```
~/workshop> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
workshop-app1-1	nginxdemos/hello	"/docker-entrypoint...."	app1	16 seconds ago	Up 15 seconds	0.0.0.0:8080->80/tcp
workshop-app2-1	nginxdemos/hello	"/docker-entrypoint...."	app2	16 seconds ago	Up 15 seconds	80/tcp

app2 is only reachable from within the network: **expose**

Useful commands

```
~> docker compose up      # Starts the project in foreground
~> docker compose up -d   # Starts the project in the background
~> docker compose ps      # Shows all running services
~> docker compose down    # Stops the project
~> docker compose logs    # Shows the logs of all containers
~> docker compose pull    # Pulls the images of all services
```


Break



Scaling your Application



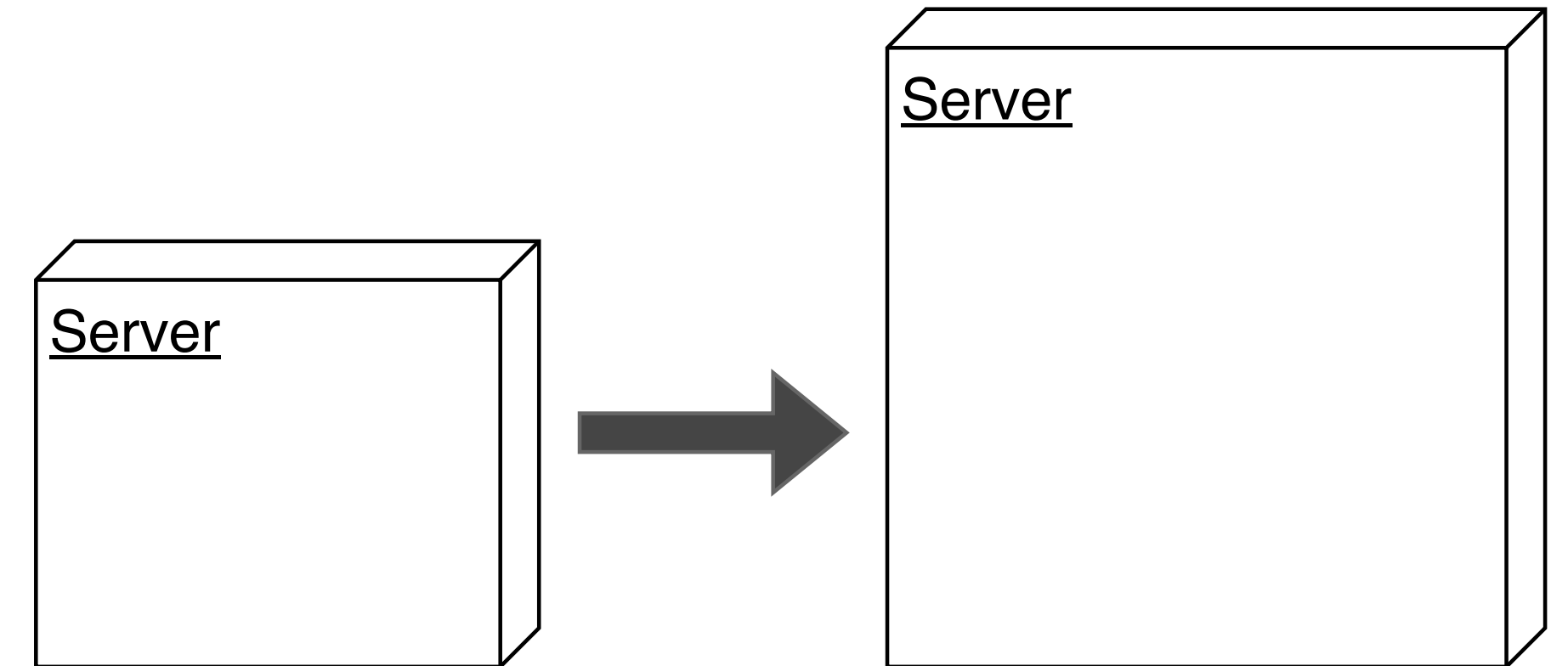
What is scaling?

Scalability is the property of a system to handle a growing amount of work by adding resources to the system.

- How can we scale?
 - Make the existing resources more capable
 - Add more of the same resources we have already

Scale up vs. Scale out

- **Vertical Scaling:** Scaling up / down
 - Scale the machine (More CPU / More RAM)
 - **Benefits**
 - No software changes required
 - Easy to maintain
 - **Drawbacks**
 - Limited by physics 🙄
 - Hardware cost may be significant



Scale up vs. Scale out

- **Horizontal Scaling:** Scaling out / in

If we talk about scaling today we mean horizontal scaling

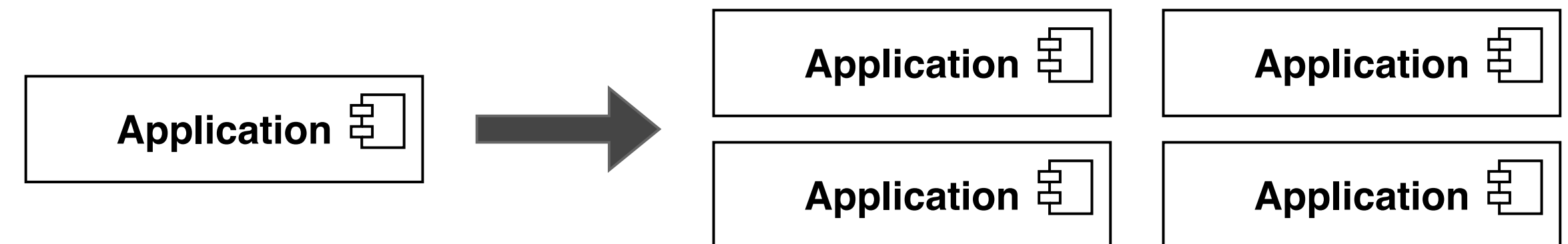
- Scale the application

- **Benefits**

- Flexible
- Limitless (More or less 😊)

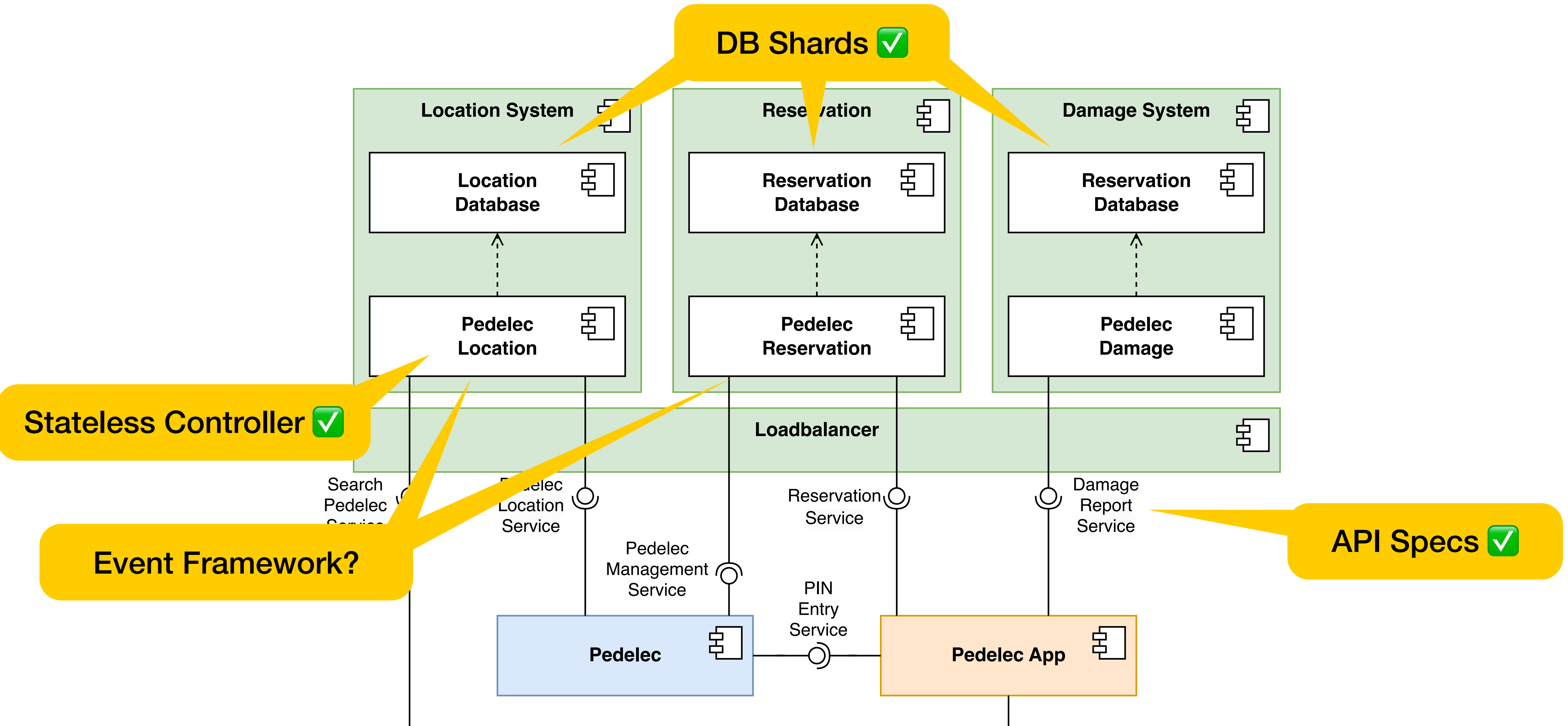
- **Drawbacks**

- More complex architecture
- Harder to debug



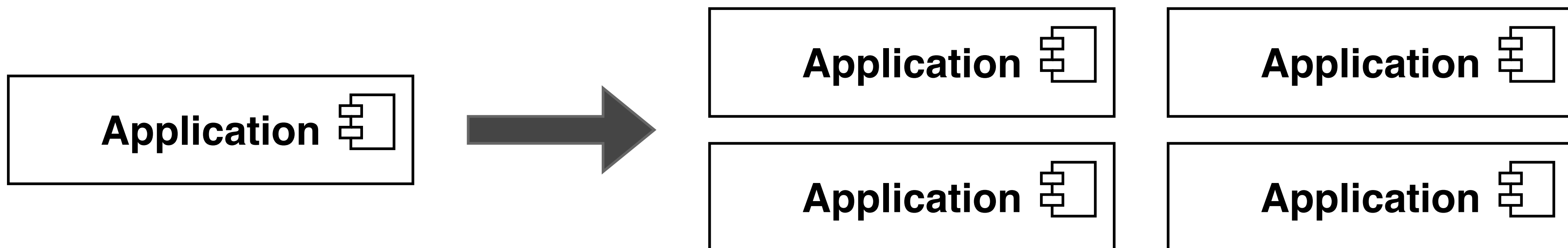
- Key Aspects for a scalable architecture
 - **API-First Design:** Clear interfaces between components.
 - **Stateless Services:** Design services to avoid local state
 - **Async Communication:** Prefer message queues or event-driven designs
 - **Database Sharding/Replication:** Avoid bottlenecks by scaling data layers
 - **Observability and Monitoring:** Integrate logging, metrics, and tracing
 - **Resilience and Fault Tolerance:** Design for failure with retries, circuit breakers, and graceful degradation

Pedelec Server Side



Problems with scaling

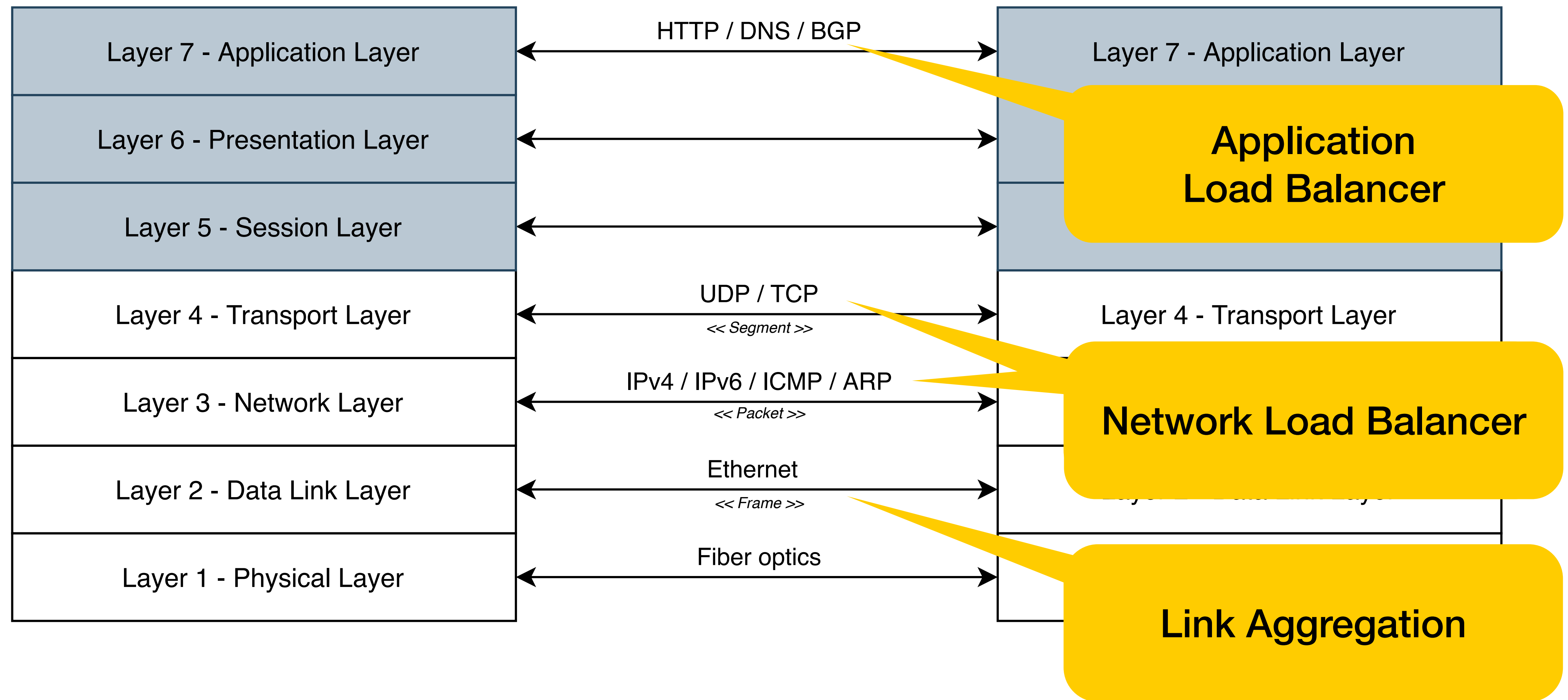
- Service Discovery
- Load Balancing
- Shared Data
- Shared Storage
- Configuration Management
- Observability



Load balancing is the process of distributing a set of tasks over a set of resources (computing units), with the aim of making their overall processing more efficient.

- Load balancing can be conducted on **different levels**
- Example: University Assignments
 - Distribute by exercise sheet
 - Distribute by exercise
 - Distribute by sub problem

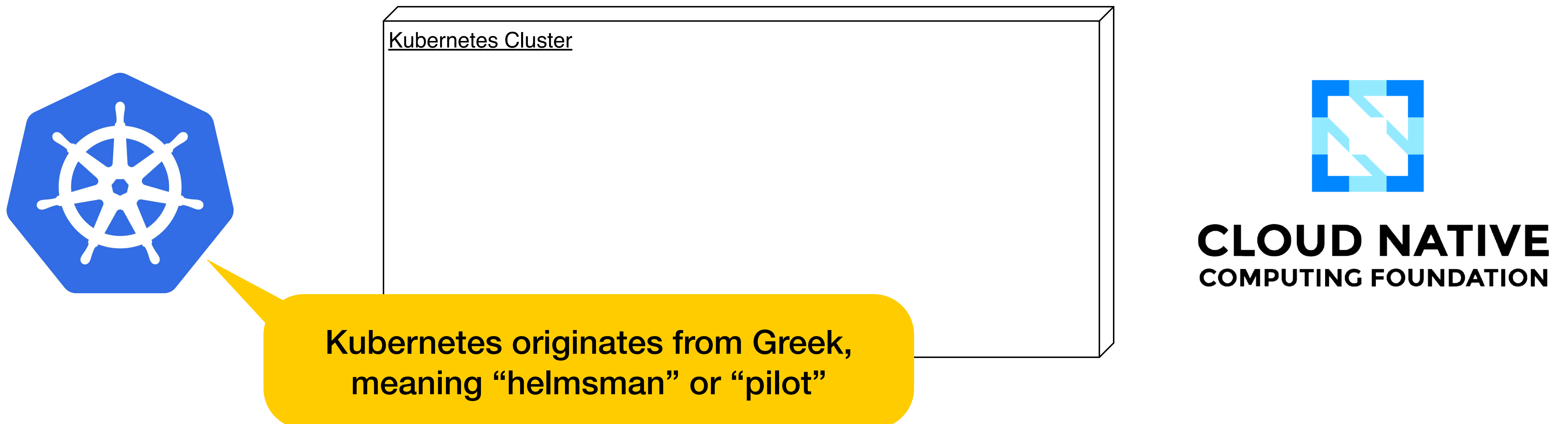
ISO/OSI Model



Kubernetes (K8s)

What is Kubernetes?

Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications.

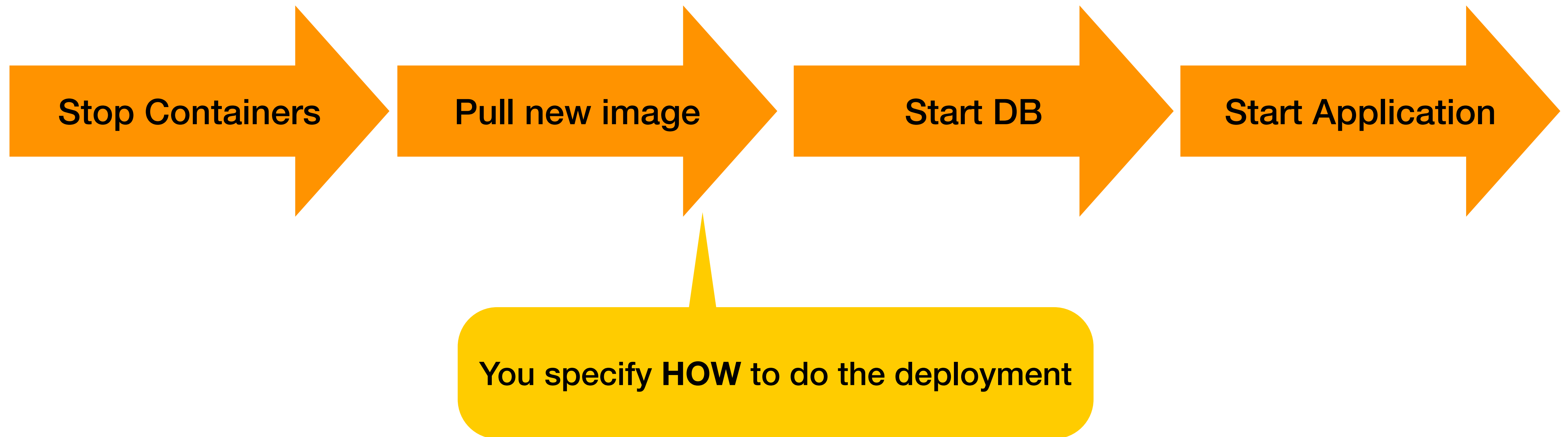


Why Kubernetes?

-  Infrastructure Abstraction
-  Automatic horizontal scaling
-  Automatic workload scheduling
-  Automated rollouts and rollbacks
-  Service discovery and load balancing
-  Storage orchestration
-  Secret and configuration management
-  Self-healing
-  Fully API driven

K8s is fundamentally different!

- **Traditional** Continuous Delivery Pipeline

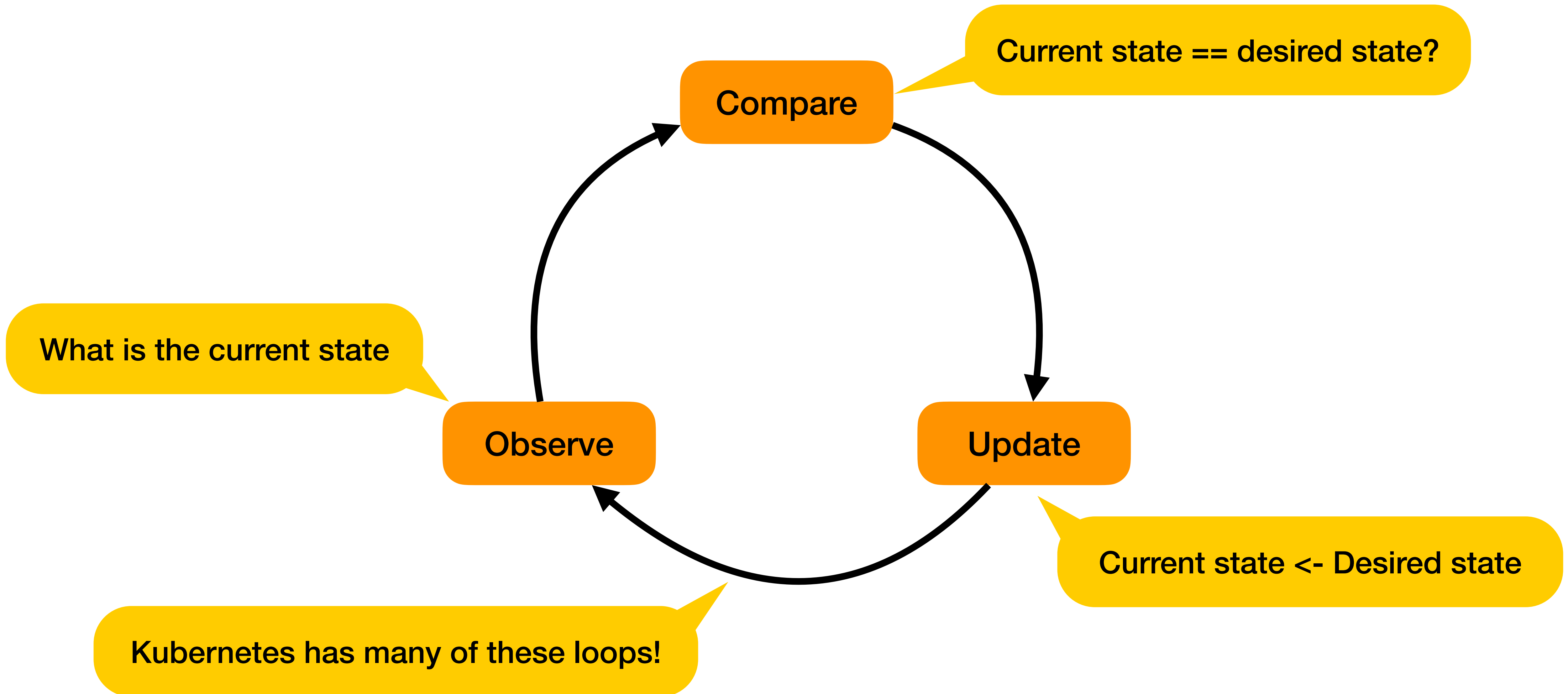


K8s is fundamentally different!

- **Declarative** Deployment Workflow
- State a Wishlist
 - “I want version 1.2.3 running”
 - “I need the containers to restart on failure”
 - “Always have 3 running instances”
- Kubernetes will try its best to fulfill your wishlist:
 - **Control Loop / Reconciliation Loop**

You specify **WHAT** should be there

Reconciliation Loop



How to Kubernetes?

- Built around **API Objects**
- Everything in K8s has a **name**!
- **Declarative** definition of state
 - “This is how I want it!”
→ Kubernetes Manifests
- CLI Tool:  **kubectl**
- GUI Tool:  **FreeLens**

```
apiVersion: <API VERSION>
kind: <API Object>
metadata:
  name: <Object Name>
  namespace: <Object Namespace>
spec:
  <Object Specification>
```

Kubernetes Management: Rancher

- What is Rancher?
 - Kubernetes **Orchestration Platform**
 - Multi-Cluster Management
- Why Rancher?
 - UI to interact with clusters
 - **Manage access** clusters
 - **Manage projects**



<https://www.rancher.com/>

Warning!

- Everything created in the Workshop Rancher Project will be deleted!
- You may use the cluster for your project at the AET group
 - iPraktikum Teams can request K8s access
 - Every DevOps Team will get a Rancher Project automatically
- The cluster has a fair use policy!
 - Only reserve what you really need
 - Make sure your code/deployment is not vulnerable
 - We will remove projects that misbehave without notice

Exercise: Get Access To Rancher



Task 1: Open rancher.ase.cit.tum.de

Task 2: Log in with your TUM ID

Task 3: Access the Student Cluster

Exercise: Kubectl



If you already have something in there -
create a backup of the file first!

Task 1: Download the `kubeconfig`

Task 2: Put the `kubeconfig` into `~/ .kube/config`

Task 3: Ensure the connection is working: `kubectl cluster-info`

Break

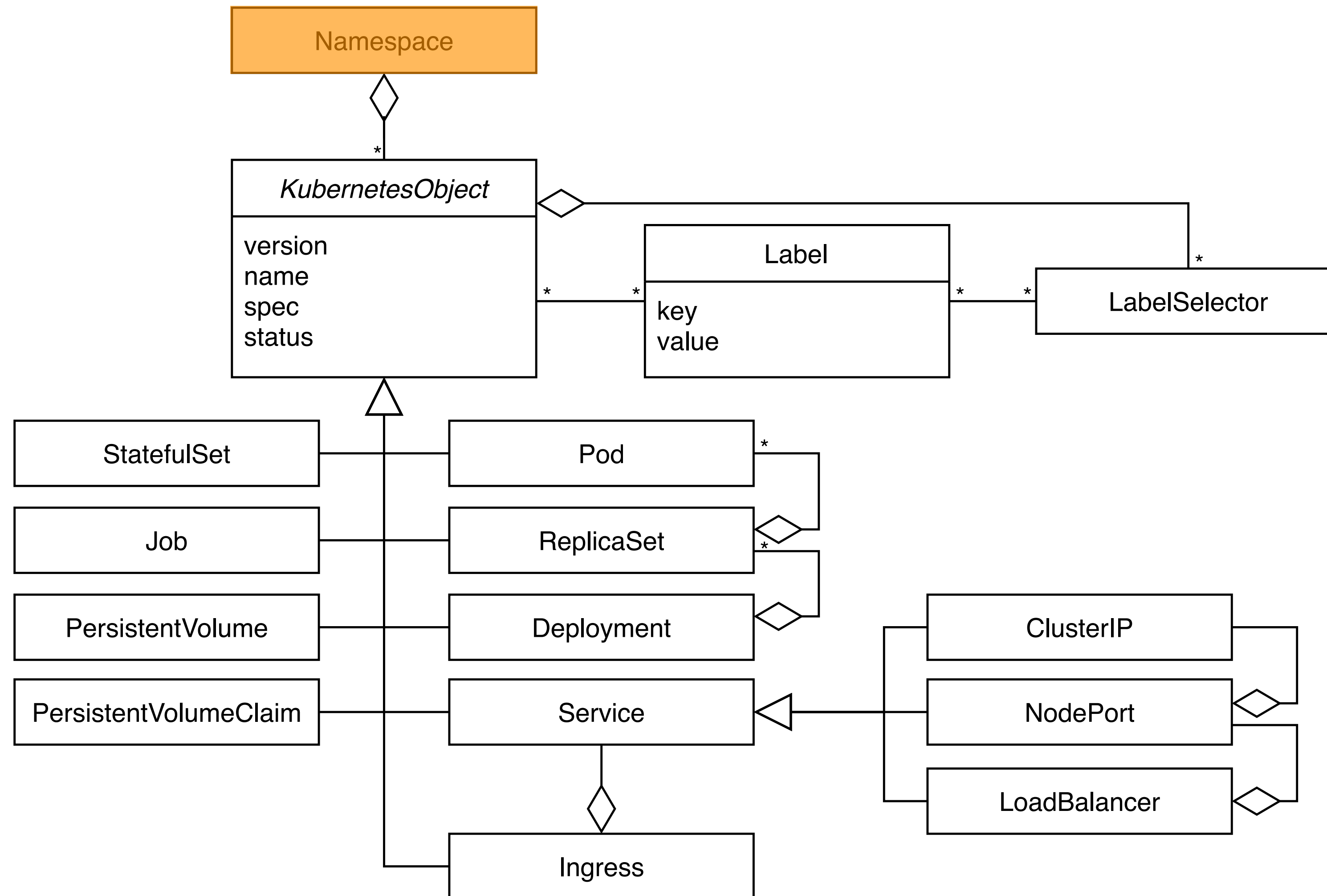


Kubernetes API and API Objects

- Namespace
- Pod
- Workloads
 - (ReplicaSet)
 - Deployment
- Service
- Ingress



Kubernetes Objects



Namespaces

- **Isolate** groups of **resources**
- Encapsulate/**scope names**
 - Names of resources within a Namespace have to be unique
- **Access Control**
- Resource Quotas

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```



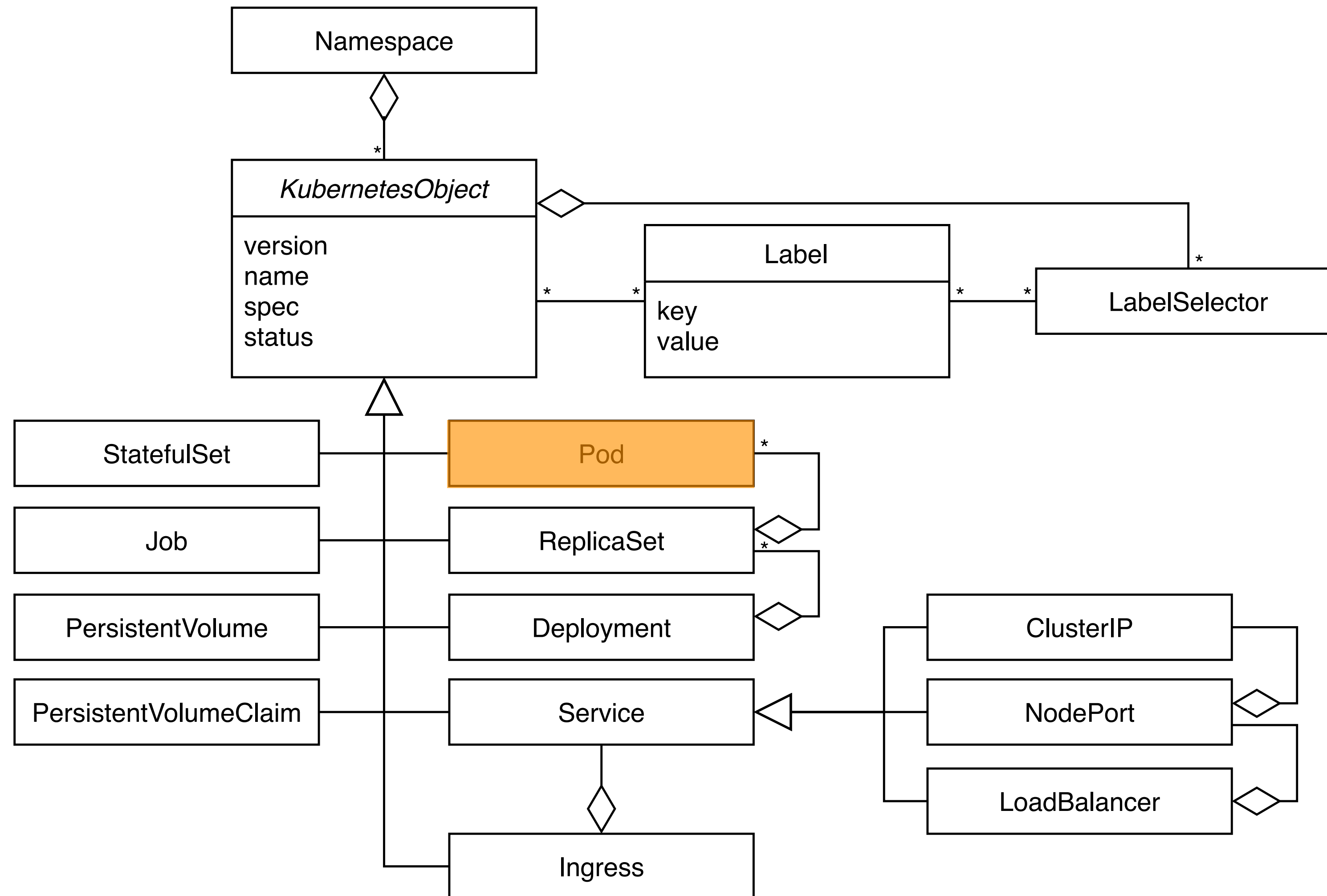

Get to know the cluster

This is a important step to work
with the Rancher Cluster

- Create a namespace
 - Open Rancher and navigate to the respective cluster
 - Select Cluster > Projects/Namespace
 - Click on the “Create Namespace” button in the respective Project
- See what is running in a specific namespace

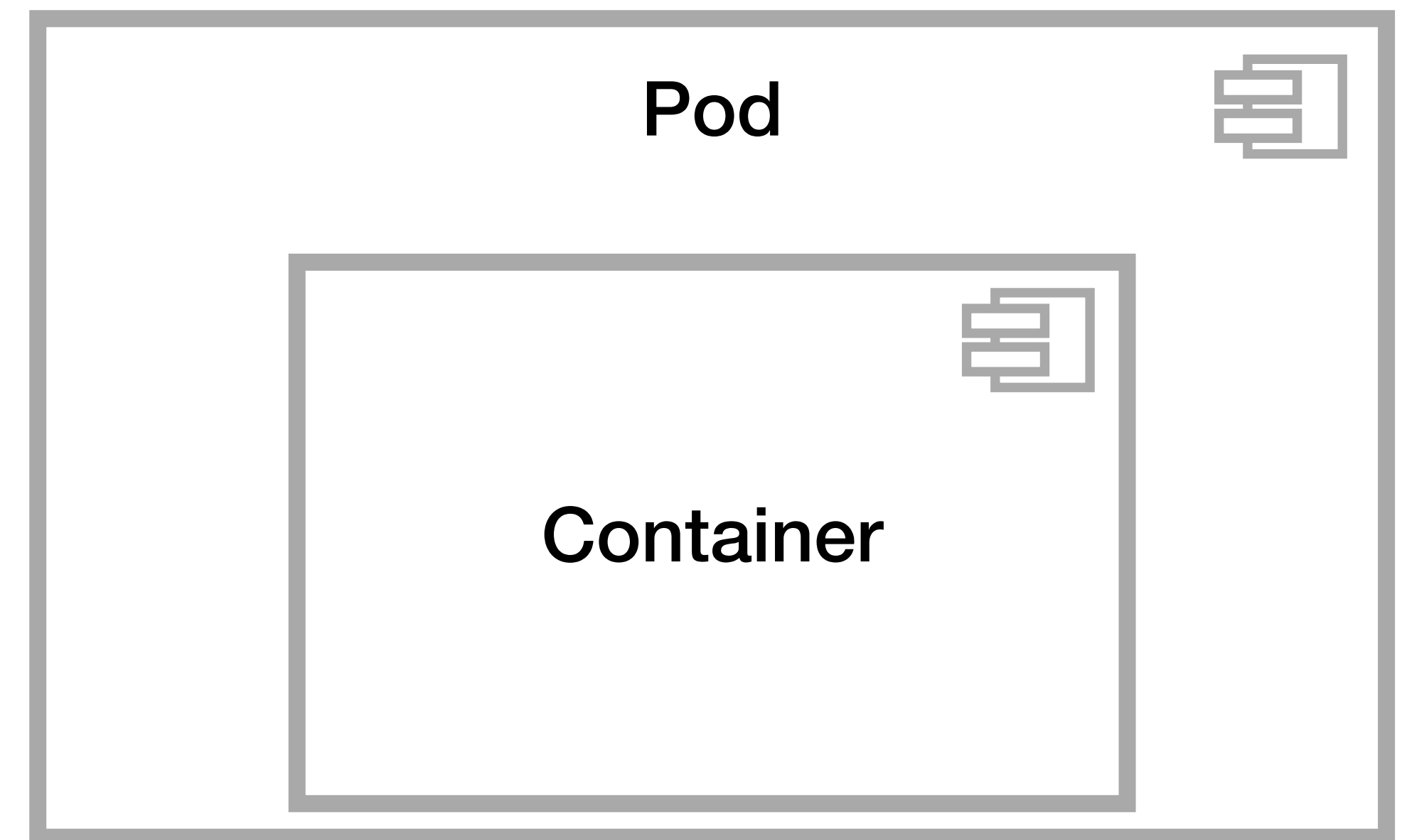
```
~/workshop> kubectl -n <namespace> get all
```

Kubernetes Objects



Pods

- **Smallest deployable computing unit** in Kubernetes
- Group of **one or more containers**
- Ephemeral nature
 - “Designed to be destroyed”
- Containers within a Pod **share**
 - Storage
 - Network (IP Address)
- It is uncommon to work with Pods directly



Mostly one container per pod

Pods

- **Smallest deployable computing unit** in Kubernetes
- Group of **one or more containers**
- Ephemeral nature
 - “Designed to be destroyed”
- Containers within a Pod **share**
 - Storage
 - Network (IP Address)
- It is uncommon to work with Pods directly

```
apiVersion: v1
kind: Pod
metadata:
  name: app1
  namespace: my-namespace
  labels:
    exampleLabel: my-app1
spec:
  containers:
  - name: app1
    image: nginxdemos/hello
    ports:
      - containerPort: 80
        name: app1-port
```

Pods

```
apiVersion: v1
kind: Pod
metadata:
  name: app1
  namespace: my-namespace
  labels:
    exampleLabel: my-app1
spec:
  containers:
  - name: app1
    image: nginxdemos/hello
    ports:
    - containerPort: 80
      name: app1-port
```

Name and Namespace of the Pod

Labels to select the Pod later on

List of containers in the Pod

Name and image of the container

List of ports to the container

Get to know the cluster

- Deploy manifest to the cluster

Remember: You are just “stating your wishlist” - not actually deploying something

```
~/workshop> kubectl apply -f example.yaml
```

- Update manifest

```
~/workshop> kubectl apply -f example.yaml
```

- Delete all resources in the manifest

```
~/workshop> kubectl delete -f example.yaml
```


Debugging in Kubernetes

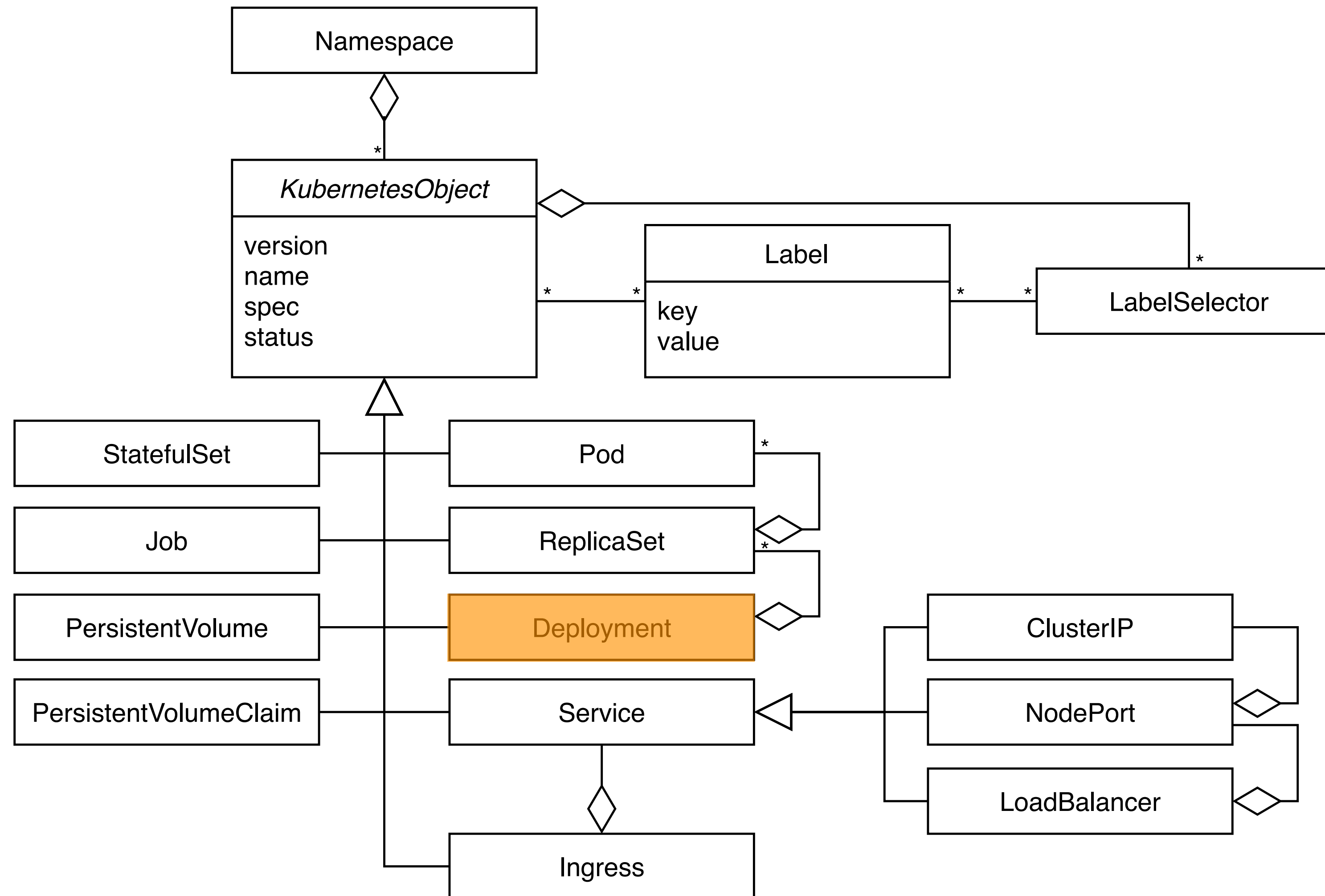
- Get a Kubernetes resource

```
~/workshop> kubectl [-n namespace] get pod
```

- Get details about a Kubernetes resource

```
~/workshop> kubectl [-n namespace] describe pod <name>
```

Kubernetes Objects



Deployments

- **Declarative control group** for Pods and ReplicaSets
 - Update images
 - Update environments
- Manages **application scaling**
- Manages **rollout** and **rollback**
- **LabelSelector** based

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1-deployment
  namespace: my-namespace
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app1-selector
  template:
    metadata:
      labels:
        app: app1-selector
    spec:
      containers:
      - name: app1-container
        image: nginxdemos/hello
```


Deployments

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1-deployment
  namespace: my-namespace
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app1-selector
  template:
    metadata:
      labels:
        app: app1-selector
    spec:
      containers:
        - name: app1-container
          image: nginxdemos/hello
```

Name and Namespace of the Deployment

n Pods the Deployment should create

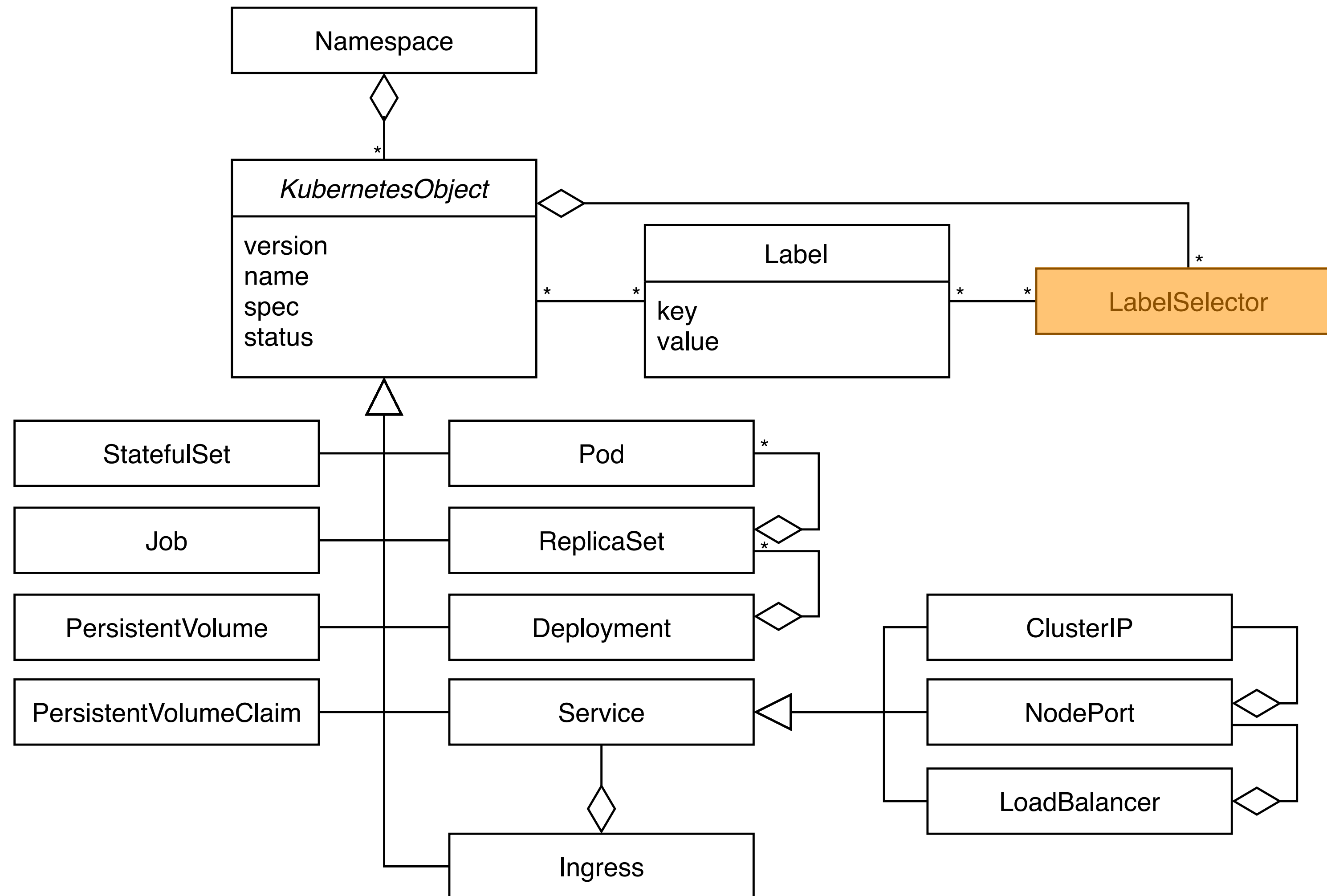
Pod selector: “Who is part of the deployment”

Template for Pods

Labels which all spawned Pods will receive

Name and image of the containers in the Pod

Kubernetes Objects



Interacting with Deployments

- Deploy a deployment manifest to the cluster

```
~/workshop> kubectl apply -f deployment.yaml
```

- Inspect the deployment

```
~/workshop> kubectl describe deployment <name>
```

- Get the application logs

```
~/workshop> kubectl logs deployment/<name>
```

- Scale the application to multiple running instances

```
~/workshop> kubectl scale deployment <name> --replicas=3
```

- Revert to the last deployed version

```
~/workshop> kubectl rollout undo deployment <name>
```


Exercise: Deployment for the Reservation System



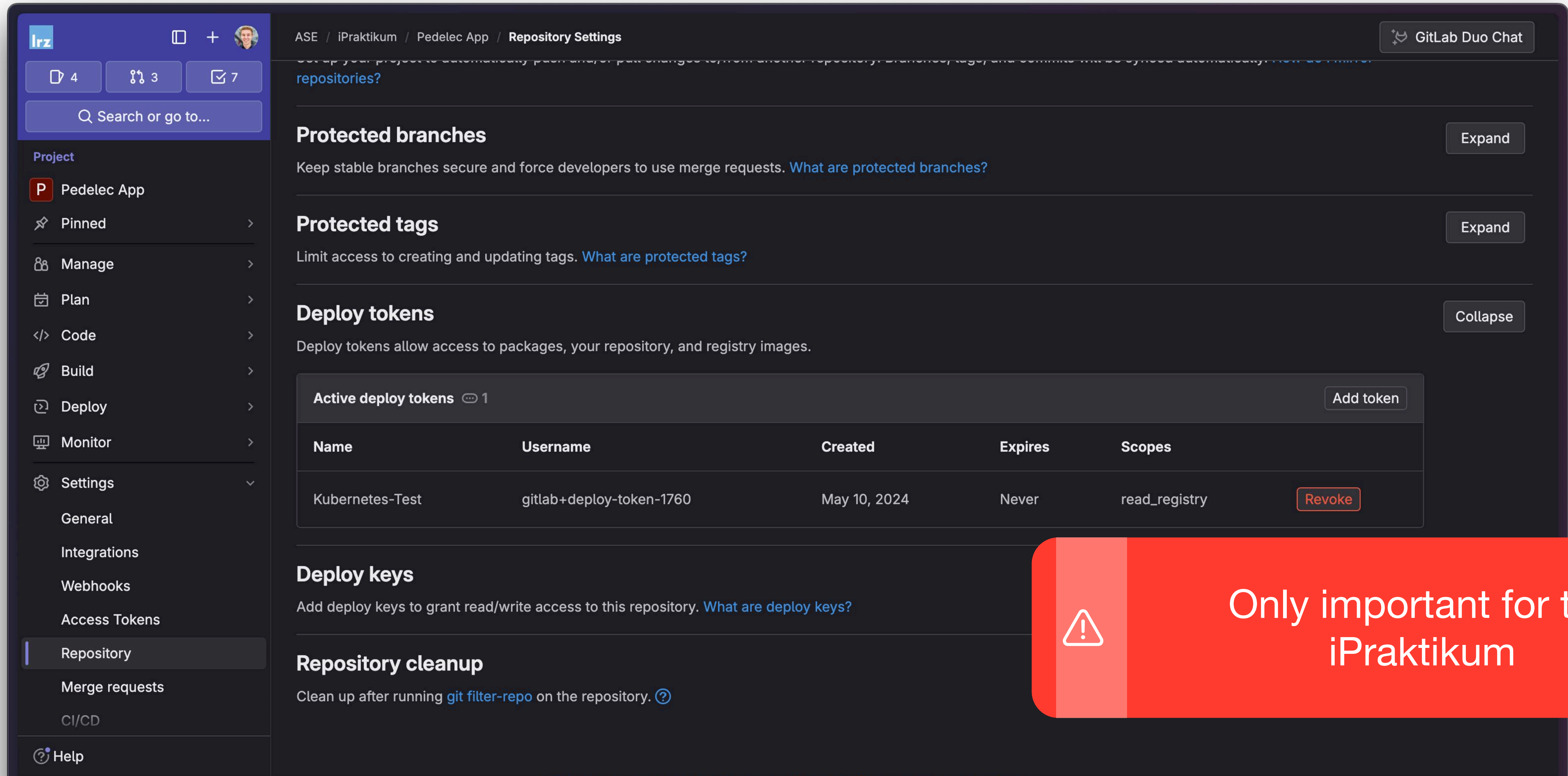
You can use the templates in your repo as a starting point

~~Task 1: Configure the Cluster to be able to pull from gitlab.lrz.de or GitHub~~

Task 2: Create a deployment manifest for the reservation system

Task 1: Configure Pull Secrets

- Create an access token to your registry



The screenshot shows the GitLab interface for the 'Pedelec App' repository settings. The left sidebar contains navigation links: Project, Pinned, Manage, Plan, Code, Build, Deploy, Monitor, Settings (General, Integrations, Webhooks, Access Tokens, Repository, Merge requests, CI/CD), and Help. The main content area is titled 'Repository Settings' and includes sections for Protected branches, Protected tags, Deploy tokens, and Deploy keys. The 'Deploy tokens' section is expanded, showing a table of active tokens.

Name	Username	Created	Expires	Scopes
Kubernetes-Test	gitlab+deploy-token-1760	May 10, 2024	Never	read_registry

A red callout box with a warning icon and the text 'Only important for the iPraktikum' is overlaid on the bottom right of the screenshot.

Task 1: Configure Pull Secrets

- Create the clone secret in your namespace

```
kubectl -n NAMESPACE create secret \
  docker-registry gitlab-lrz \
  --docker-username=USERNAME \
  --docker-password=PASSWORD \
  --docker-server=gitlab.lrz.de:5005
```

This is the “manual” way of deploying a secret -
please use manifests whenever possible



Only important for the
iPraktikum

Task 1: Configure Pull Secrets

- Change to the kubernetes-start branch
- Update the namespace in the secrets file:

```
~/.../k8s> code deployment-secrets.yml
```

- Add the clone secret to your namespace

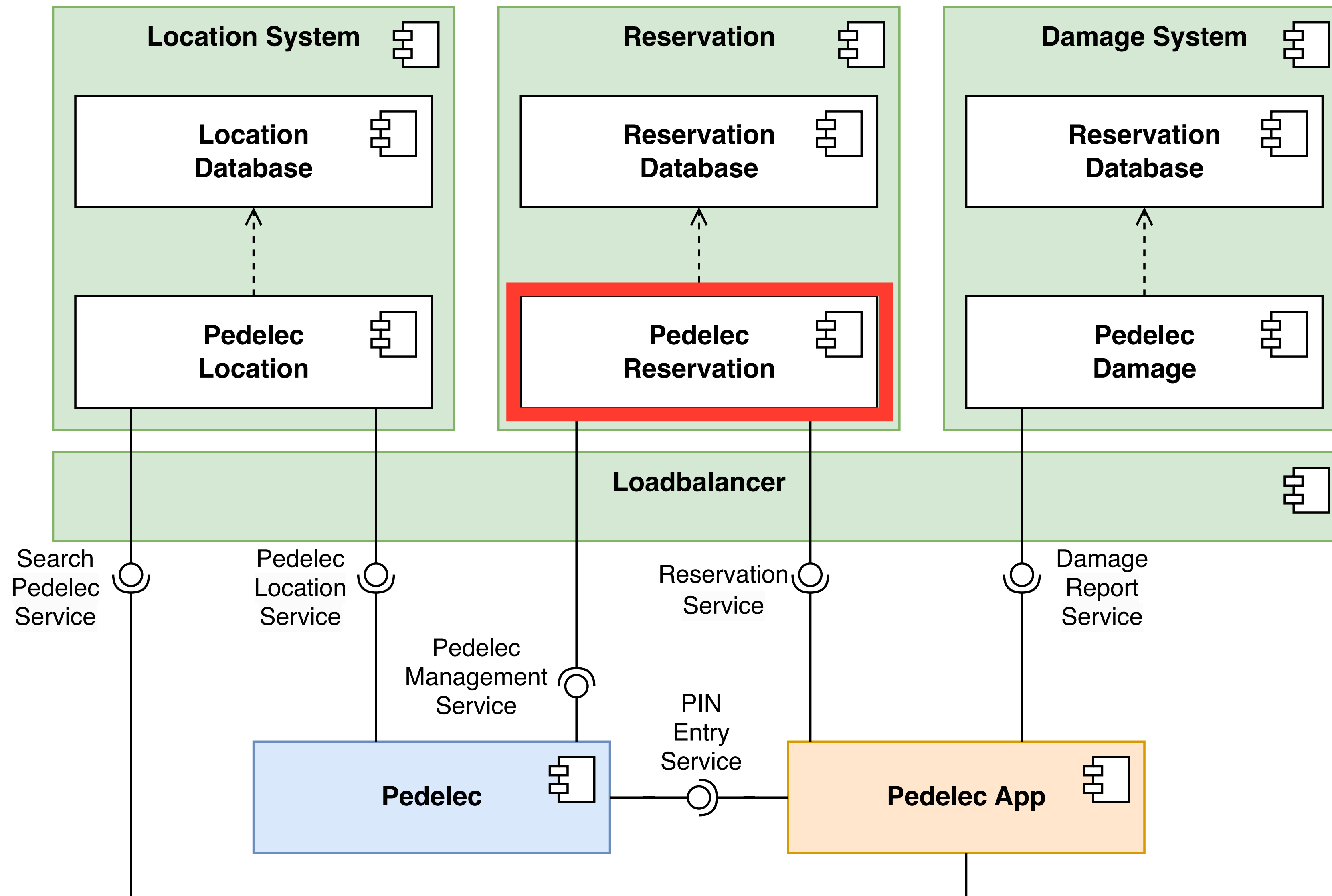
```
~/.../k8s> kubectl apply -f deployment-secrets.yml
```

- K8s can now pull the images without further intervention



Only important for the
iPraktikum

Task 2: Create Reservation deployment



Task 2: Create Reservation deployment

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reservation
  namespace: pedelec
spec:
  replicas: 1
  selector:
    matchLabels:
      app: reservation-selector
  template:
    metadata:
      labels:
        app: reservation-selector
    spec:
      containers:
      - name: reservation
        image: ghcr.io/mtze/pedelec-reservation:latest
        ports:
        - containerPort: 8080
          name: reservation-api
```

Remember to update the namespace to your own!

We omitted the resource limits here - you will see it in the demo

Task 2: Create Reservation deployment

- Apply the new manifest

```
~/workshop> kubectl apply reservation-deployment.yml
```

- See which Deployments are present in a specific Namespace

```
~/workshop> kubectl -n <namespace> get deployments
```

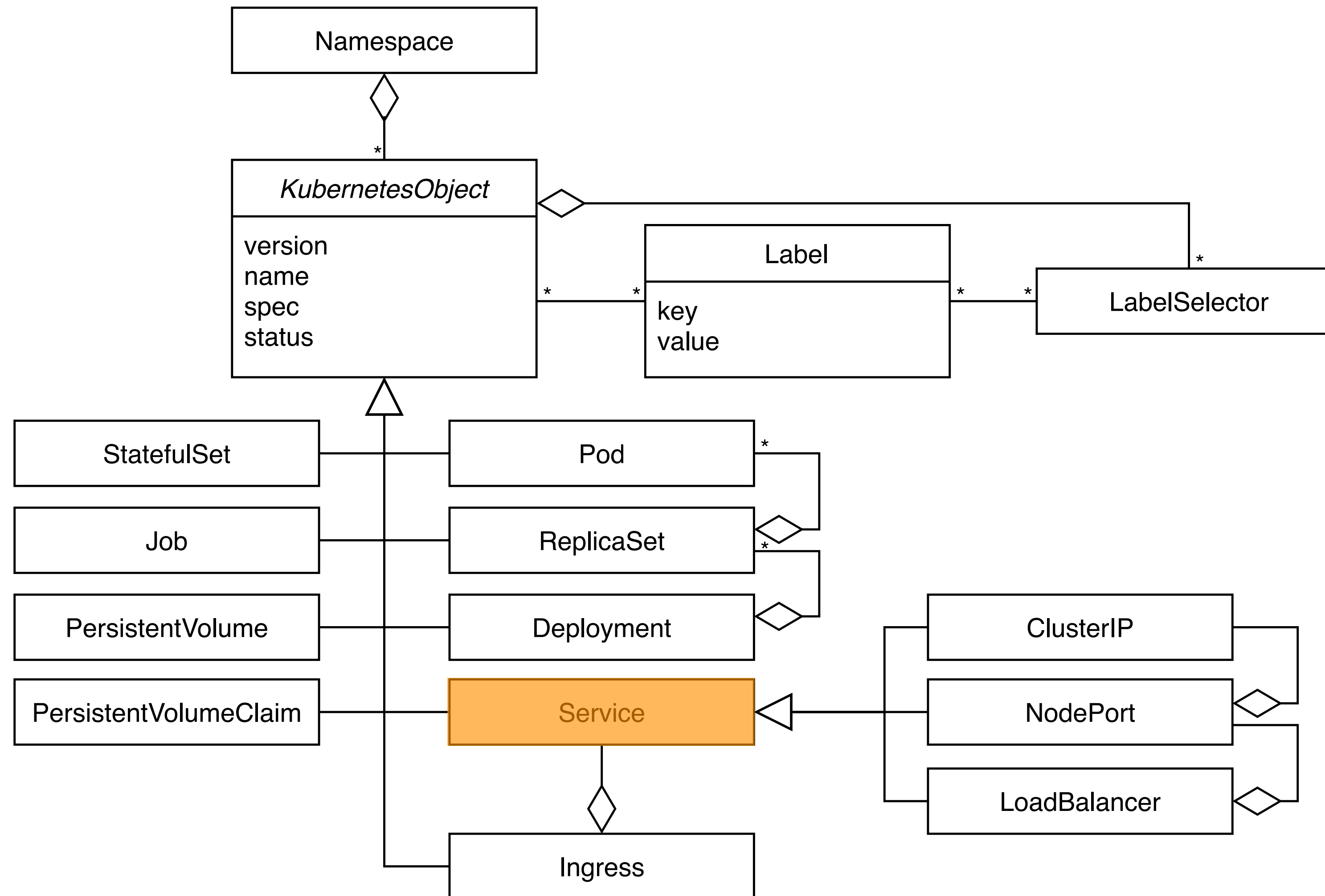
Break



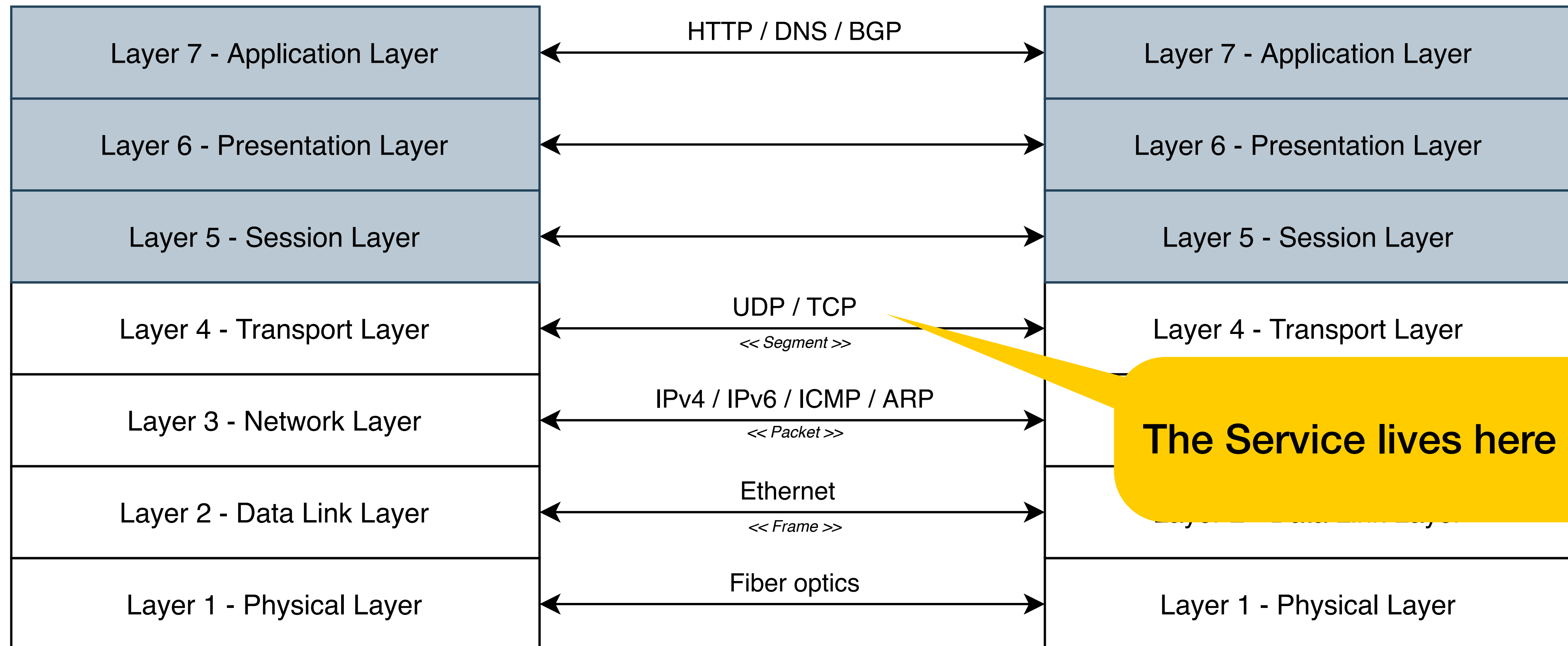
We have a problem now...

- We learned that
 - Pods are ephemeral
 - Get a new IP Address on every schedule
 - Are scattered over all Kubernetes nodes
- How do find them then?
 - -> Services

Kubernetes Objects



ISO/OSI Model



The Service lives here 🧘

Services

- Abstraction layer to expose a set of Pods in the cluster
- Manages **service discovery**
- Manages **load balancing** to Pods
- IP addressable
- **LabelSelector** based

```
apiVersion: v1
kind: Service
metadata:
  name: app1-service
  namespace: my-namespace
spec:
  selector:
    exampleLabel: app1
  ports:
    - port: 80
      targetPort: app1-port
```


Services

```
apiVersion: v1
kind: Service
metadata:
  name: app1-service
  namespace: my-namespace
spec:
  selector:
    exampleLabel: app1
  ports:
    - port: 80
      targetPort: app1-port
```

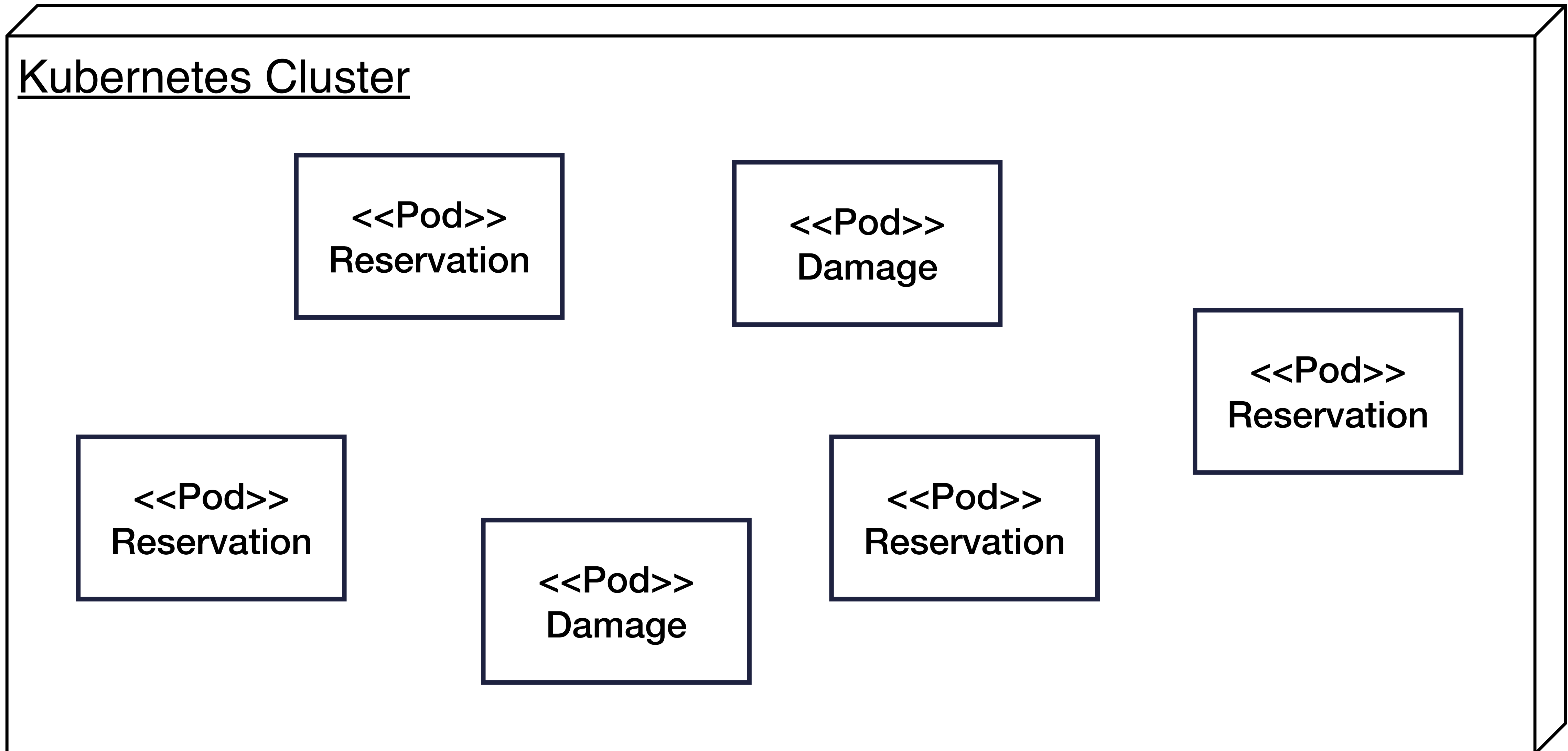
Name and Namespace of the Service

Pod selector: “Who can I send traffic to”

Port the Service exposes

Port(name) the Service connects to

Services



Services

Kubernetes Cluster

<<Pod>>
Reservation

<<Pod>>
Damage

<<Pod>>
Reservation

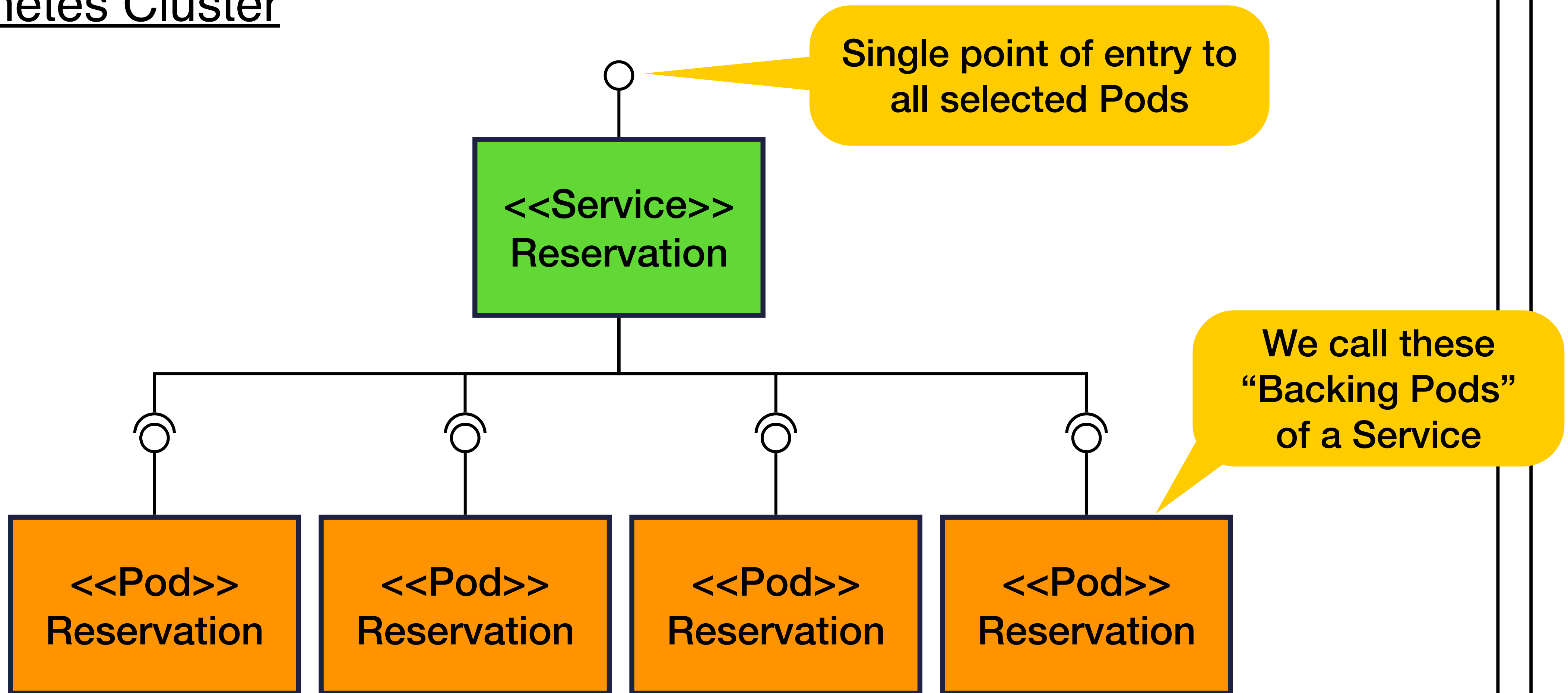
<<Pod>>
Reservation

<<Pod>>
Damage

<<Pod>>
Reservation

Services

Kubernetes Cluster



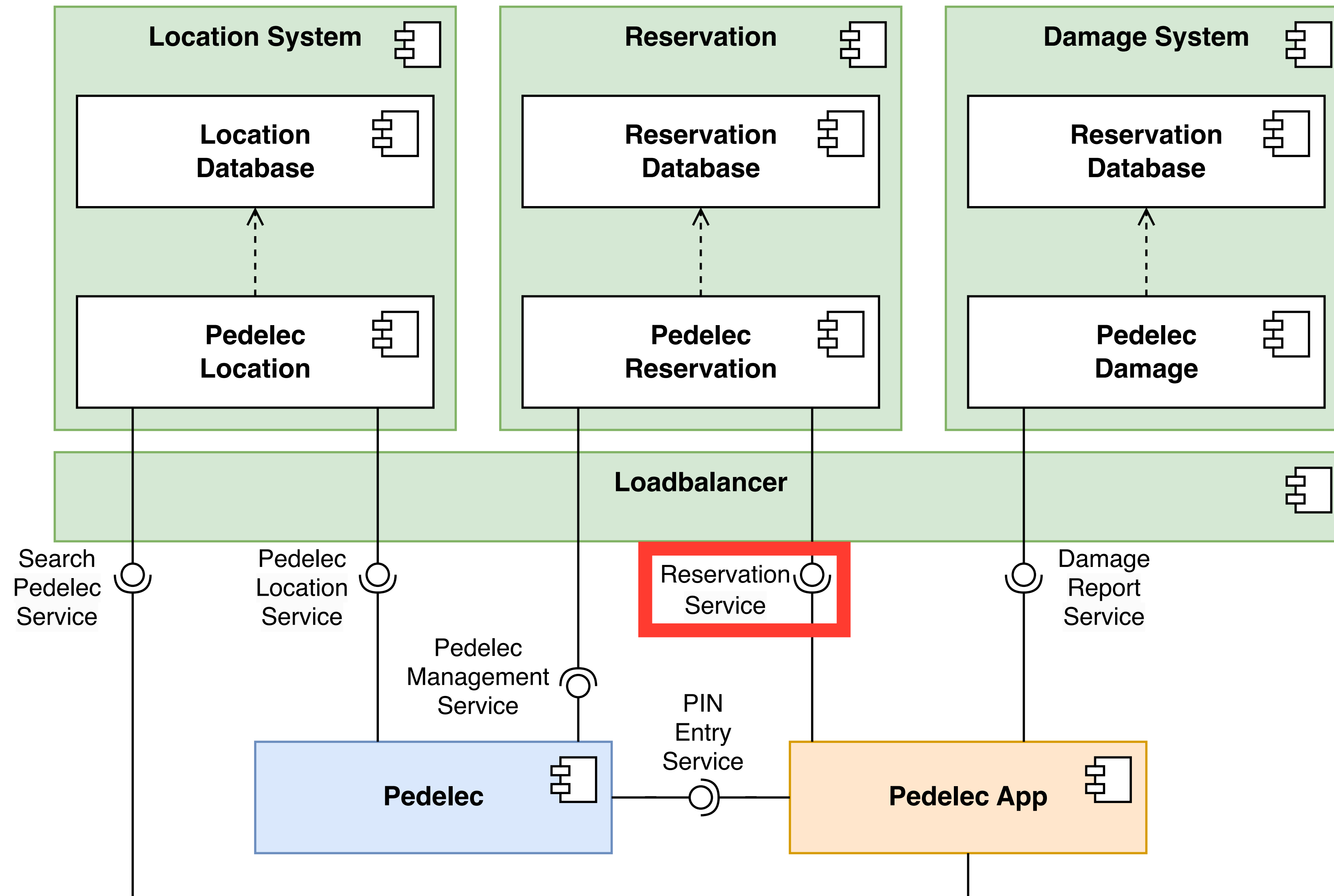
Exercise: Service for Reservation System



Task 1: Create a Service manifest for the Reservation System

Task 2: Get the Service details using `kubectl`

Task 1: Create reservation service



Task 1: Create reservation service

```
apiVersion: v1
kind: Service
metadata:
  name: pedelec-reservation-service
  namespace: pedelec
spec:
  selector:
    app: reservation
  ports:
    - port: 80
      targetPort: reservation-api
```

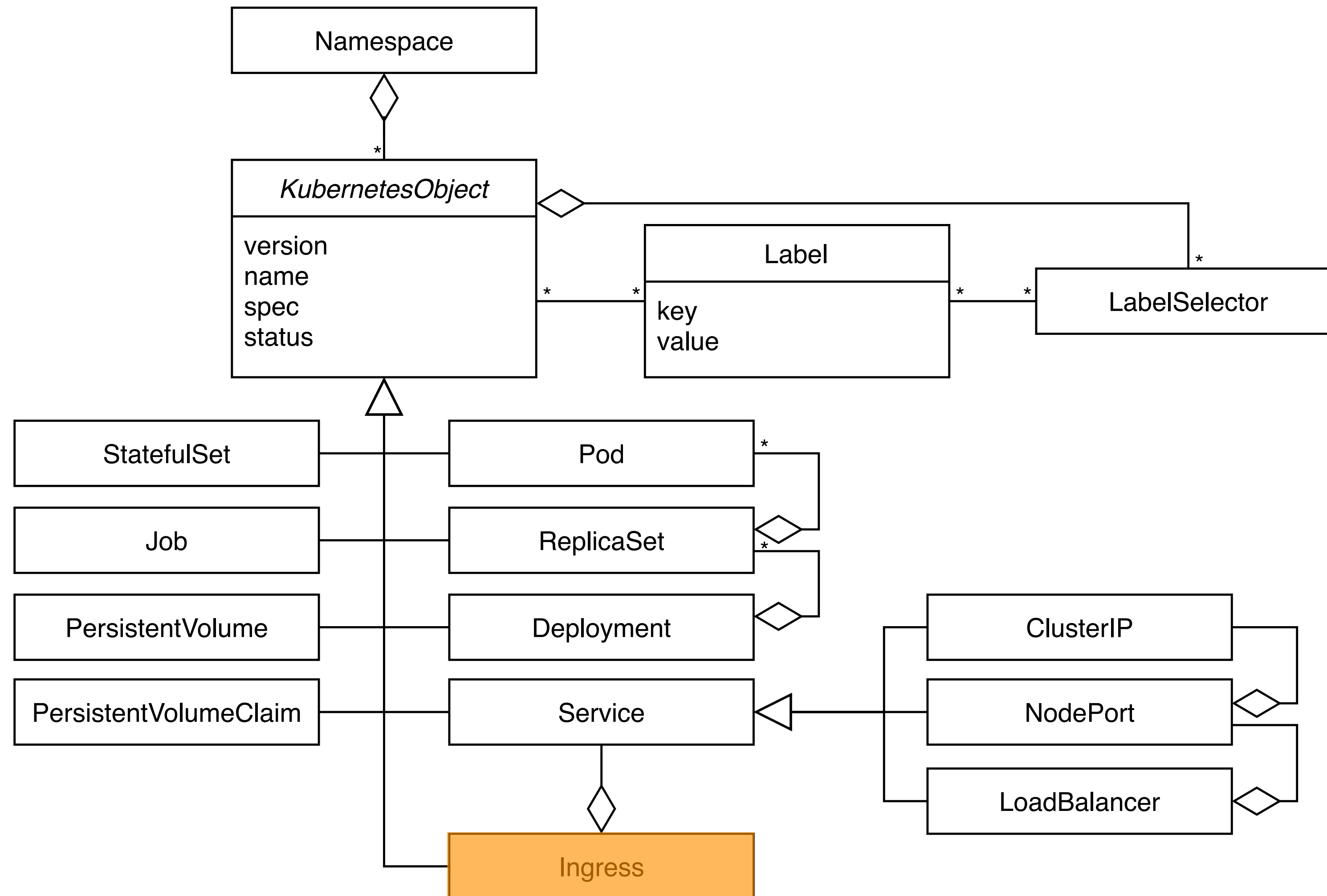
We use the same label to select the matching Pods

Task 2: Find the service

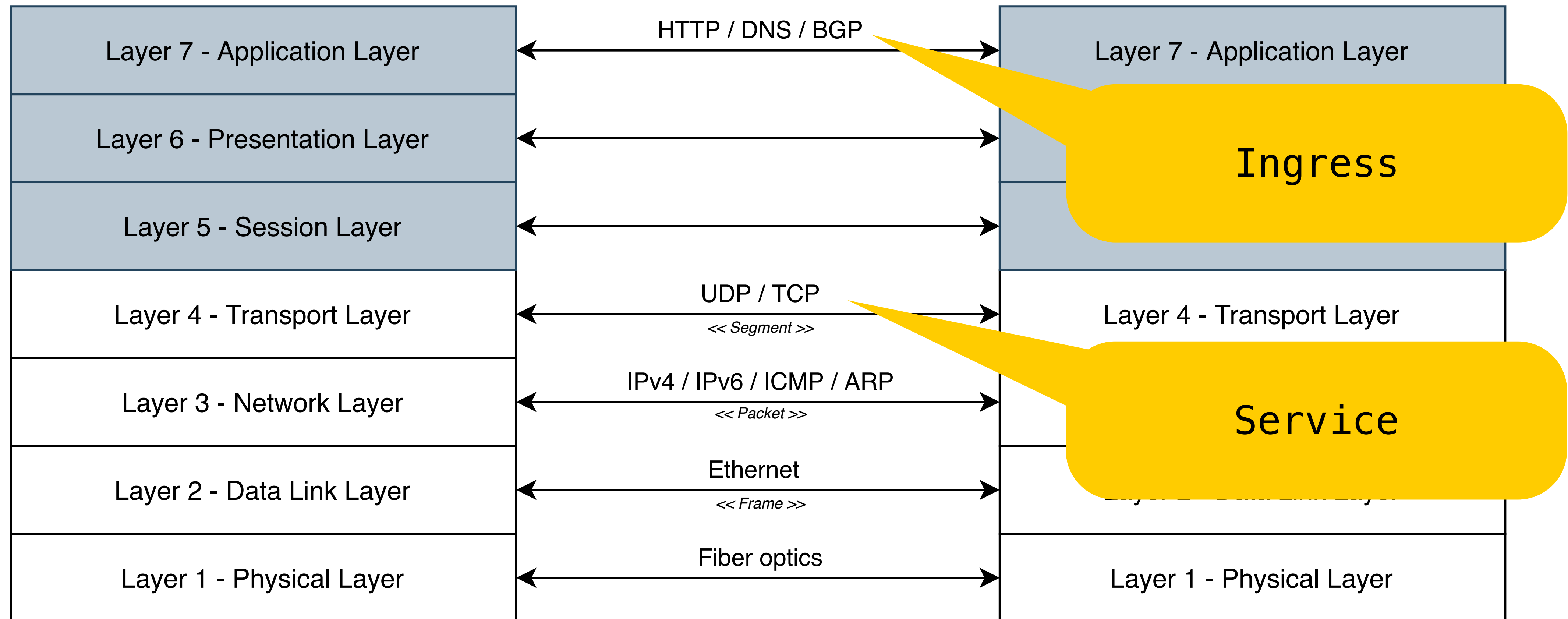
- See which Services are present in a specific namespace

```
~/workshop> kubectl -n <namespace> get services
```

Kubernetes Objects



ISO/OSI Model



Kubernetes Ingresses

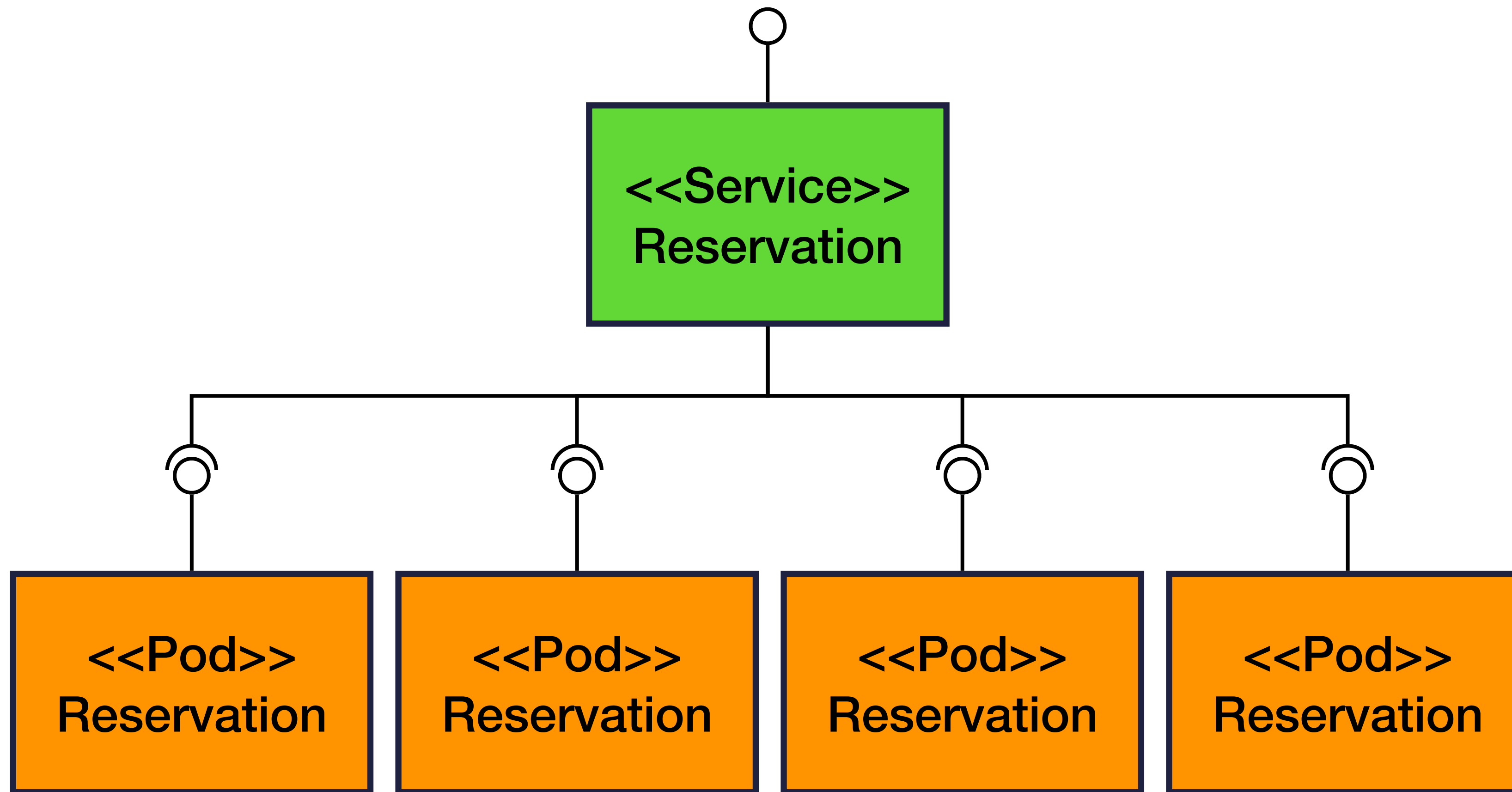
An Ingress is a Kubernetes API object that manages external access to services in a cluster, typically via HTTP/HTTPS.

- **Central entry point** for multiple services (reverse proxy)
- Clean URL routing (e.g., /api, /admin)
- **TLS termination** (HTTPS)
- Path-based and host-based routing
- An ingress needs to be implemented by a **IngressController**

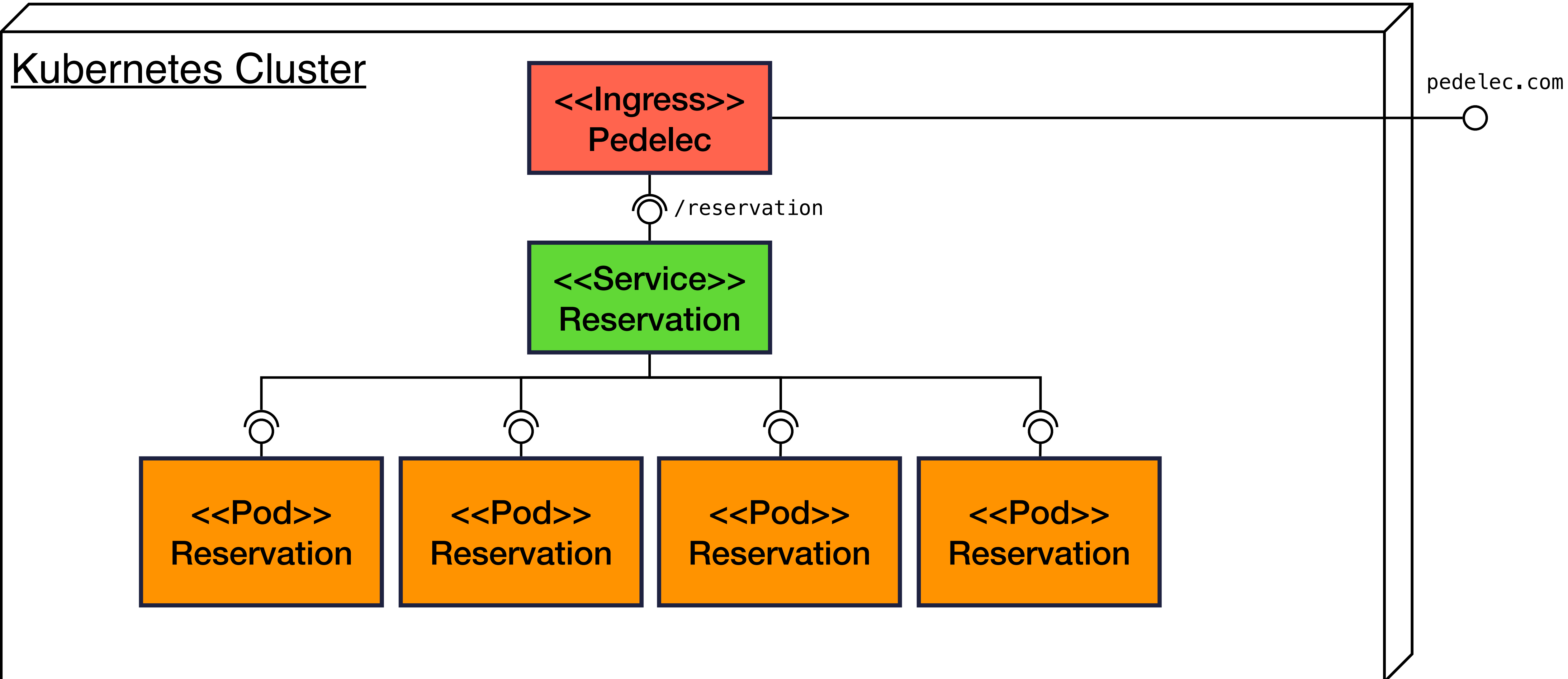
First API Object which is not in the K8s core

Services

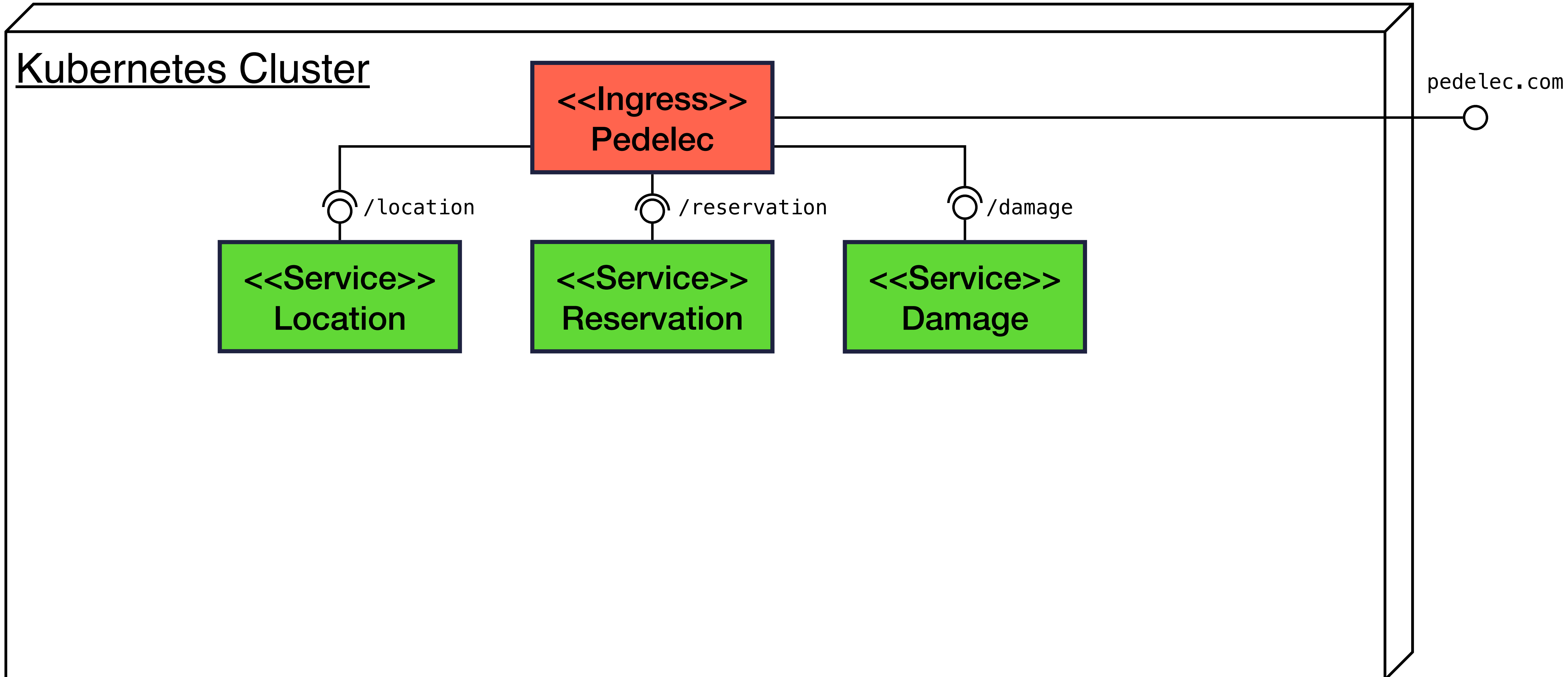
Kubernetes Cluster



Services



Services



Ingress

- **Exposes** Services outside the cluster via **HTTP/HTTPS**
 - HTTP load balancing
 - TLS termination
- Backed by an IngressController
- Acts like a “Reverse Proxy”

```
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app1-ingress
  namespace: my-namespace
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1-service
            port:
              number: 80
```


Ingress

apiVersion: networking.k8s.io/v1

kind: Ingress

metadata:

name: app1-ingress

namespace: my-namespace

annotations:

nginx.ingress.kubernetes.io/rewrite-target: /

spec:

rules:

- host: example.com

http:

paths:

- path: /

pathType: Prefix

backend:

service:

name: app1-service

port:

number: 80

Notice the different apiVersion

Name and Namespace of the Ingress

New concept! Annotations:
arbitrary non-identifying metadata

Hostname the ingress listens for

Path the ingress routes

Service and Port of the backing Service

Exercise: Ingress for the Pedelec Server



Task 1: Single Ingress deployment

Task 2: Test the endpoints with bruno

Task 3: Add remaining endpoints to ingress

Important Nots

- You can **only use** *.stud.k8s.aet.cit.tum.de in the cluster by default
- You need to make sure that the backing pod **expects the correct route**
- **Ingress configurations** are heavily based on annotations
 - You can **request a TLS certificate** via an Annotation
 - Make sure to test with staging first ;)

Lets Encrypt has strict load balancing policies

```
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app1-ingress
  namespace: my-namespace
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: letsencrypt-staging
spec:
  tls:
  - hosts:
    - pedelec.com
    secretName: reservation-tls
  rules:
  - host: pedelec.com
    http:
      paths:
      - path: /reservation
        pathType: Prefix
        backend:
          service:
            name: pedelec-reservation-service
            port:
              number: 80
```


From Compose to K8s

- Keep in mind:
 - **This mapping is not general advice but for our example it is sufficient!**
- Mapping:

Docker World		Kubernetes World
Service	→	Deployment
Expose	→	Service
Port	→	Service / Ingress



Docker and Kubernetes use the word “Service” to describe different things!

Break



Kubernetes Packaging: Helm

What is a Helm Chart

Helm Charts help you define, install, and upgrade even the most complex Kubernetes application. Charts are easy to create, version, share, and publish — so start using Helm and stop the copy-and-paste.

- Three+ Concepts

Remember the hassle changing the namespace?

- The **chart** is a bundle of information necessary to create an instance of a Kubernetes application.
- The **values** contains configuration information that can be merged into a packaged chart to create a releasable object.
- A **release** is a running instance of a chart, combined with a specific config.

Helm Motivation

- **Reusable & Shareable:** Charts help version, reuse, and distribute your deployments
→ One chart can deploy to dev, staging, and production with different values
- **Multi-Environment Support:** Easily manage Test, Staging, and Production via separate values.yaml files
- **Cloud Portability:** Abstract infrastructure specific configuration – move from GCP to AWS with minimal changes
- **Simple Installation:** Install off-the-shelf apps (e.g., Kafka, Grafana, NextCloud) with one command

What is an Helm Chart?

```
~/workshop/helm> tree
```

```
my-chart/
```

```
|— Chart.yaml      # Metadata (name, version, description)
|— values.yaml     # Default configuration values
|— templates/      # Templated Kubernetes Manifests / YAML files
└— charts/         # Optional dependencies
```

- Parameterized Kubernetes Manifests
- Filled with values on install / upgrade
- Optional Helper functions

The Helm value concept

```
# values.yml
```

```
---
```

```
replicaCount: 2
```

```
image:
```

```
  repository: myregistry/pedelec
```

```
  tags: v1.2.0
```

```
# Deployment Template
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
spec:
```

```
  replicas: {{ .Values.replicaCount }}
```

```
  template:
```

```
    spec:
```

```
      containers:
```

```
      - name: app1-container
```

```
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Notice - we no longer specify the namespace

These Values are placed in the on install

Helm Getting Started

- Create a chart skeleton

```
~> helm create pedelec
```

- Install a chart (release)

```
~> helm -n <namespace> install <release-name> <chart>
```

- Update a release

```
~> helm -n <namespace> upgrade <release-name> <chart>
```


Exercise: Create a Helm Chart for the Pedelec Server



Task 1: Initialize the helm chart

Task 2: Copy over the manifests

Task 3: Remove namespace from all resources

Task 4: Replace hard coded values with Templates

Useful commands

```
~> helm install my-app ./my-chart      # Starts the project
~> helm install my-app -f values.dev.yaml ./my-chart # Deploy with dev config
~> helm install my-app -f values.prod.yaml ./my-chart # Deploy with prod config
~> helm uninstall my-app # Uninstalls the release and deletes resources
~> helm upgrade my-app ./my-chart # Shows the logs of all containers
~> helm template my-app ./my-chart # Prints the manifest without installing
~> helm lint ./my-chart # Validates chart structure and templates for errors or best practices
```

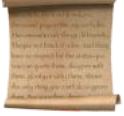
Summary

Post Survey



<https://go.tum.de/397452>

Summary

-  What **Kubernetes** is and why it's the backbone of modern infrastructure
-  Manifests – the declarative way to define your cluster's desired state
-  Building Blocks of a **Cluster**
 -  Pods – the smallest deployable units
 -  Deployments – manage replicas and rolling updates
 -  Services – stable endpoints for dynamic workloads
 -  Ingresses – routing external traffic into the cluster
-  Namespaces – organize and control environments and teams cleanly
-  **Helm** – the package manager for Kubernetes

Summary

- What we achieved today:
 - You deployed the pedelec server project to the cloud! 🚀



..but where to go from here?

Further Materials and Topics

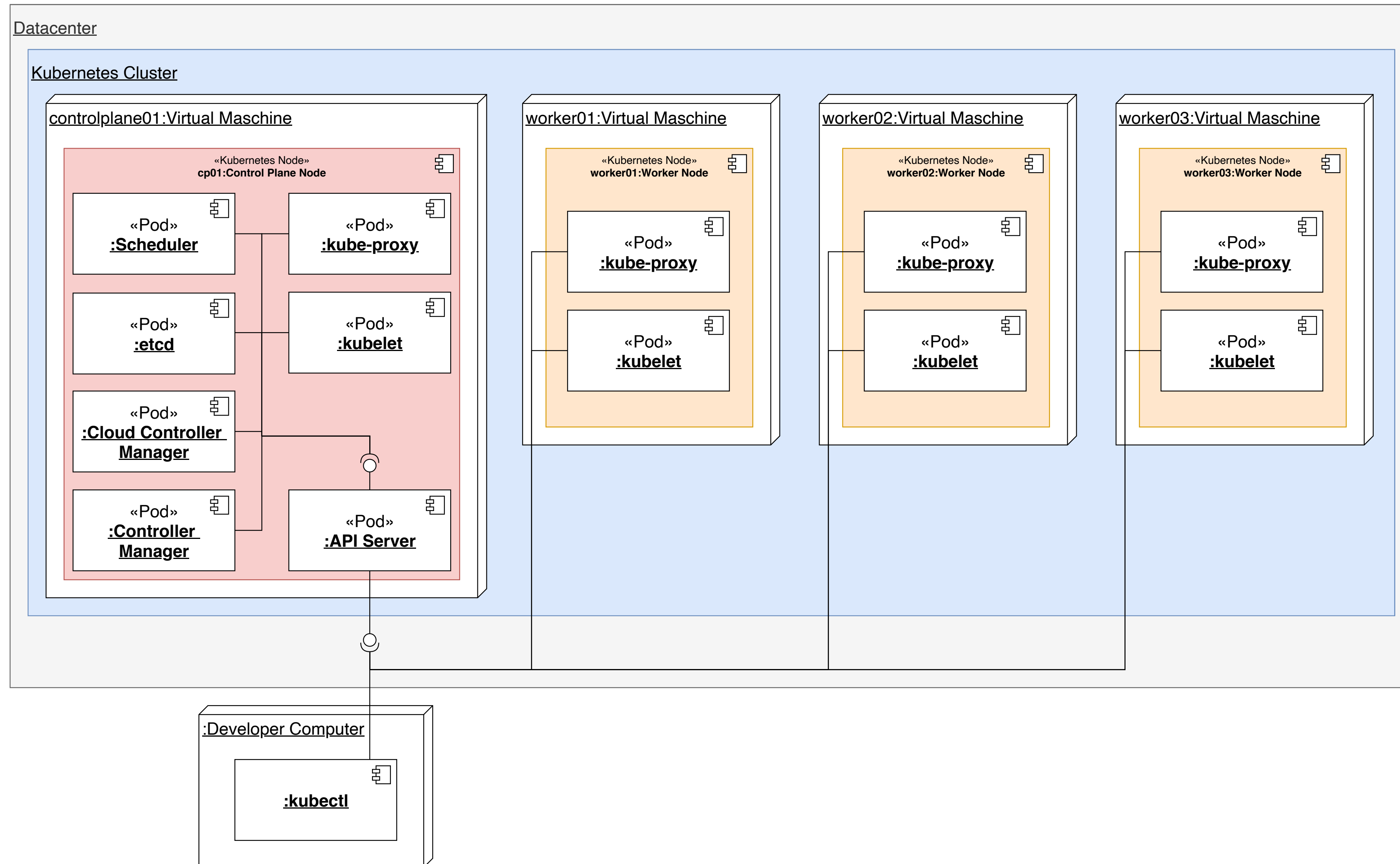
- Developer side
 - Make it big: **Autoscalers**
 - Work with ENV Variables: **ConfigMaps** and **Secrets**
 - Deep Dive: **Labels** and **LabelSelectors**
 - Persistent Storage: **Volumes** and **VolumeClaims**
- Kubernetes Internals
 - Cluster Architecture
 - Control Plane
 - Container Lifecycle
 - CRI, CNI, CSI

Resources

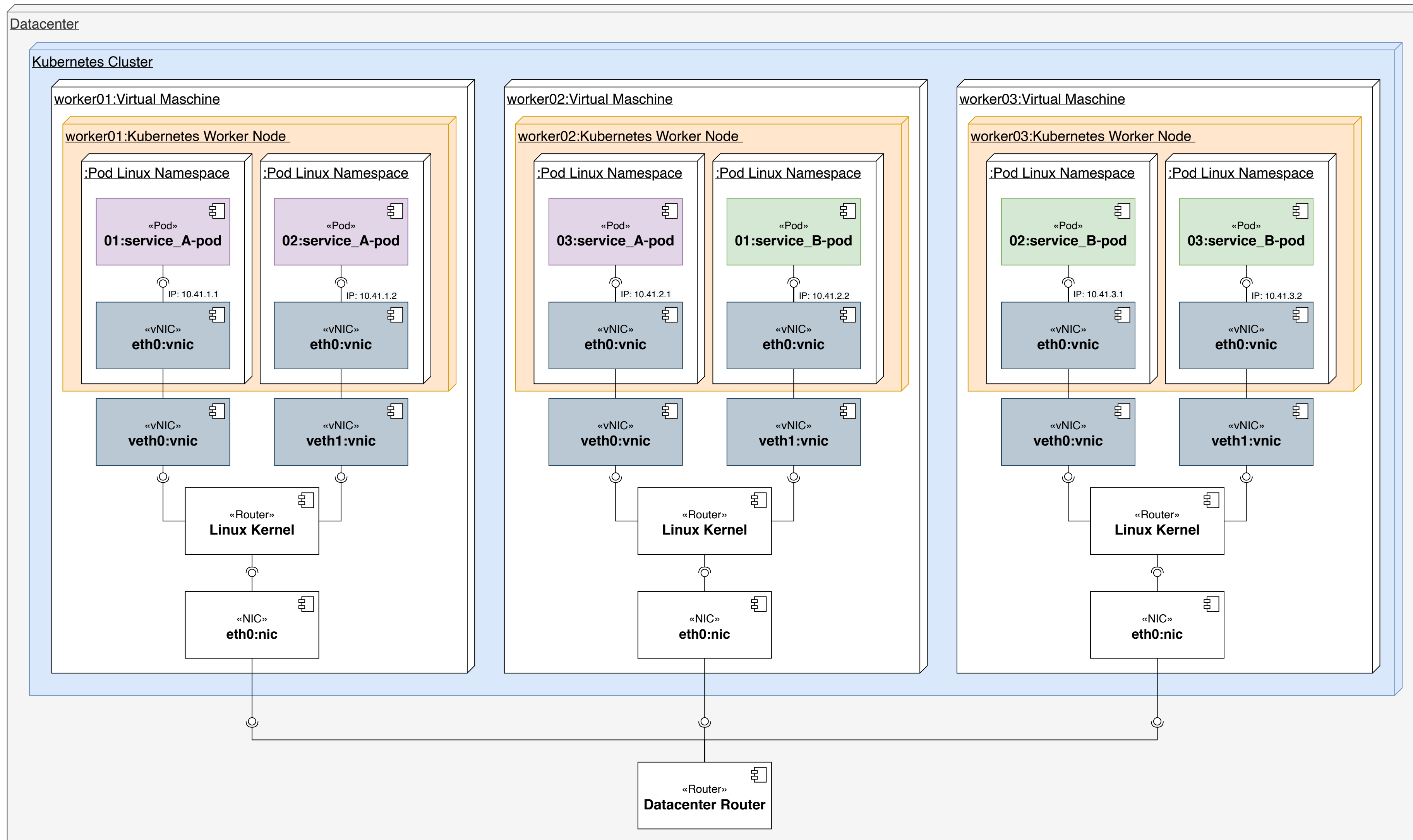
- Docker
 - <https://docs.docker.com/compose/>
 - <https://docs.docker.com/compose/compose-file/>
- Kubernetes
 - <https://kubernetes.io/docs>
- Youtube
 - <https://youtu.be/iVj5vEnbFr0?si=Z4XJyKV4hvs50k4K>

[Optional] Kubernetes under the hood

Control Plane



Kubernetes Networking



Kubernetes Storage

